

DeutschLingoDeck - Documentation

1. Business Overview

1.1 Purpose of the Document

This document defines the software architecture and technical design of DeutschLingoDeck, a web-based card training application built with Angular, Spring Boot and PostgreSQL.

The purpose of this document is to establish a clear architectural foundation before implementation begins. It describes the main system components, responsibilities, design principles, data model, backend structure, frontend structure, security model, API design, deployment approach and future extensibility options.

The document is intended to serve as the primary technical reference for the development of the application. It should guide implementation decisions, reduce architectural ambiguity and ensure that future extensions are aligned with the original design.

1.2 Scope

The scope of this document includes the complete application architecture for the first production-oriented version of the system.

The system consists of three main technical components: an Angular single-page application, a Spring Boot REST API and a PostgreSQL database. These components are designed to run together inside a Docker Compose environment, while still following principles that would allow the system to be deployed in a production-like environment.

The document covers user management, dictionary import, card management, game sessions, answer validation, statistics tracking, authentication, authorization, database design, REST API design, Docker-based deployment, logging, monitoring, testing and future roadmap considerations.

The document does not provide a full implementation of the application. Instead, it defines architectural rules, design decisions, domain models and technical guidelines that should be followed during implementation.

1.3 System Goals

The primary goal of DeutschLingoDeck is to provide a structured card-based training system where users can upload their own dictionaries, start training sessions, answer cards and receive statistics based on their own learning history.

The system must support multiple users. Each user must be able to manage only their own dictionaries, games, answers and statistics. User data isolation is a core architectural requirement.

The system must record every game session and every answer given by the user. This event history is used as the source of truth for statistics, progress tracking and future analytical features.

The application must be designed in a way that allows future extension toward advanced features such as spaced repetition, AI-based answer evaluation, personalized learning paths, speech recognition and richer statistics.

1.4 Target Audience

This document is intended for developers, technical leads, software architects and future maintainers of the project.

Backend developers should use this document to understand the Spring Boot module structure, transaction boundaries, database model, REST API design and security rules.

Frontend developers should use this document to understand the Angular application structure, routing model, component responsibilities, state management approach and API integration principles.

Database and DevOps-oriented contributors should use this document to understand PostgreSQL schema design, migration strategy, Docker Compose configuration and operational concerns.

1.5 Architectural Approach

DeutschLingoDeck follows a production-oriented architecture even though the initial deployment may be simple. The system is intentionally designed as a modular monolith rather than a set of microservices.

The application domain is cohesive: users upload dictionaries, train with cards, submit answers and receive statistics. These operations are strongly related and often require

transactional consistency. Splitting the system into microservices at this stage would introduce unnecessary operational and architectural complexity.

The backend is therefore implemented as a single Spring Boot application, internally divided into clearly separated feature modules. Each feature module owns its domain logic and exposes functionality through well-defined service boundaries.

The frontend is implemented as an Angular single-page application using standalone components, feature-based structure and Angular Signals for local reactive state.

The database is implemented in PostgreSQL and managed through versioned schema migrations.

1.6 Key Architectural Principles

The architecture is based on the following principles.

Separation of concerns: each layer and module has a clearly defined responsibility. Controllers handle HTTP communication, services handle business logic, repositories handle persistence, and DTOs define API contracts.

Feature-based modularity: the backend and frontend are organized around business features instead of purely technical layers. This improves maintainability and keeps related code close together.

Stateless backend: the backend does not depend on server-side HTTP sessions. Authentication is handled through tokens, which allows the backend to remain horizontally scalable.

Database as source of truth: persistent user data, dictionaries, games, answers and statistics are stored in PostgreSQL. Frontend state is temporary and never replaces backend persistence.

Event history before aggregation: individual game answers are stored as historical records. Aggregated statistics are derived from these records and can be reconstructed if needed.

Security by default: all user-owned resources must be accessed through ownership-aware queries. A user must never be able to access another user's dictionaries, games, answers or statistics.

API-first communication: the frontend communicates with the backend exclusively through REST APIs. The Angular application must never access the database directly.

Controlled extensibility: the architecture should be simple enough for the initial version but structured enough to support future features without major redesign.

1.7 Document Structure

The following chapters describe the system in increasing technical detail.

The first part defines the business context, system goals, architectural principles and major architecture decisions.

The second part describes the domain model, database design, backend architecture and frontend architecture.

The third part covers cross-cutting concerns such as authentication, authorization, error handling, logging, testing, Docker deployment and monitoring.

The final part describes future roadmap options and possible extensions of the system.

2. Business Overview

2.1 Business Context

DeutschLingoDeck is a web application designed to support structured vocabulary and sentence learning through interactive flashcard-based training sessions. Unlike traditional language learning platforms that rely on predefined content, the system allows each user to build and manage their own learning material by importing dictionaries from Microsoft Excel files.

The application focuses on providing a personalized learning experience where every user owns their dictionaries, learning sessions and progress statistics. Each interaction with the system contributes to a continuously growing learning history, allowing future improvements such as adaptive learning algorithms and AI-assisted tutoring.

The system is intentionally designed as a generic learning platform rather than a language-specific application. Although the first version focuses on Croatian-to-German translation exercises, the underlying architecture does not depend on any specific language pair. Future versions should support additional source and target languages without requiring architectural changes.

2.2 Business Problem

Many learners maintain personal vocabulary lists in spreadsheets because existing learning platforms often provide limited flexibility regarding custom content.

These spreadsheets quickly become difficult to maintain as the number of words grows. They typically lack structured statistics, learning history, progress tracking and intelligent repetition strategies. As a result, users have little visibility into which vocabulary has already been mastered and which requires additional practice.

DeutschLingoDeck addresses these limitations by transforming static spreadsheets into an interactive learning platform while preserving the user's ability to maintain their own learning material.

The system separates dictionary management from the learning process. Dictionaries become reusable data sources from which users can start multiple independent learning sessions while continuously accumulating long-term statistics.

2.3 Business Objectives

The primary business objectives of the application are:

- Allow users to create and manage their own dictionaries.
- Support importing dictionaries from a predefined Excel template.
- Provide fast and intuitive flashcard-based learning sessions.
- Record every answer submitted by the user.
- Build detailed learning statistics over time.
- Enable users to monitor their personal progress.
- Prepare the platform for future adaptive learning algorithms.
- Maintain complete ownership and isolation of user data.

The system should remain easy to use while internally maintaining a rich historical dataset suitable for future analytical features.

2.4 Functional Requirements

The first production version of the application shall provide the following capabilities.

User Management

- User registration.
- Secure authentication.
- User profile management.
- Secure logout.
- Password change functionality.

Dictionary Management

- Upload dictionaries using an Excel template.
- Validate uploaded files before import.
- Display validation errors with row-level precision.
- Store imported dictionaries.
- Display dictionary details.
- Soft-delete dictionaries.
- Allow each user to manage only their own dictionaries.

Game Management

- Select one of the available dictionaries.
- Start a new learning session.
- Present flashcards one by one.
- Accept user input.
- Validate each submitted answer.

- Immediately persist every answer.
- Finish or abandon a game session.
- Display a game summary.

Statistics

- Record every completed answer.
- Track per-card statistics.
- Track per-dictionary statistics.
- Track complete game history.
- Display learning progress.
- Display success rates.
- Allow future implementation of advanced spaced repetition algorithms.

2.5 Non-Functional Requirements

The application should satisfy the following quality attributes.

Maintainability

The codebase should be organized into clearly separated feature modules with well-defined responsibilities.

Scalability

Although initially deployed as a modular monolith, the architecture should allow individual modules to be extracted into separate services if future requirements justify such a transition.

Reliability

Every game answer must be stored reliably.

The system should never lose user progress due to frontend refreshes or unexpected browser termination.

Security

All user-owned resources must remain fully isolated.

Authentication and authorization must follow modern security practices.

Passwords must never be stored in plaintext.

Uploaded files must be validated before processing.

Performance

Dictionary import should complete within a few seconds for typical dictionary sizes.

Game interactions should feel instantaneous.

Statistics retrieval should remain efficient even after years of accumulated learning history.

Extensibility

The architecture should support future implementation of:

- AI-assisted answer validation
- alternative accepted answers
- speech recognition
- text-to-speech
- adaptive learning algorithms
- spaced repetition
- additional learning modes
- multiple languages

without requiring significant architectural redesign.

2.6 Core Business Entities

The application revolves around six primary business entities.

User

Represents an authenticated person using the system.

A user owns dictionaries, games, answers and statistics.

No data is shared between users.

Dictionary

A user-created collection of learning cards imported from an Excel document.

A dictionary acts as the source material for future learning sessions.

Card

Represents one learning item.

Depending on its type, a card may represent:

- a word
- a noun
- a verb
- a phrase
- a complete sentence

Each card belongs to exactly one dictionary.

Game

Represents one learning session.

A game references a single dictionary and contains all answers submitted during that session.

Answer

Represents one user response to one card during one game.

Answers form the immutable event history of the application.

Statistics are derived from answers rather than replacing them.

Statistics

Represents aggregated information calculated from historical answers.

Statistics are maintained per user and per dictionary.

Individual card statistics describe how successfully a user has learned each specific card over time.

2.7 Business Rules

The following business rules define the core behavior of the system.

A user may own multiple dictionaries.

A dictionary belongs to exactly one user.

A card belongs to exactly one dictionary.

A game is always associated with exactly one dictionary.

Every answer belongs to exactly one game.

Every answer references exactly one card.

Every answer is permanently stored.

Deleting a dictionary must never remove completed game history.

Users may only access their own dictionaries, games, answers and statistics.

Statistics are calculated exclusively from the user's own learning history.

The application never mixes statistics between different users.

Each uploaded dictionary is independent.

Learning progress accumulated in one dictionary must not influence statistics of another dictionary.

2.8 Future Business Vision

The initial version establishes a solid technical foundation for a much broader learning platform.

Future versions may introduce:

- intelligent card scheduling
- AI-generated explanations for incorrect answers
- pronunciation evaluation
- voice-based learning
- collaborative dictionary sharing
- dictionary marketplace
- multiple language pairs
- AI-generated practice sentences
- grammar exercises
- mobile applications

- offline learning
- synchronization across multiple devices

The current architecture is intentionally designed to accommodate these future capabilities while keeping the initial implementation focused and maintainable.

3. System Overview

3.1 Architectural Overview

DeutschLingoDeck follows a classical three-tier architecture consisting of a presentation layer, an application layer and a persistence layer.

The system is intentionally designed as a modular monolith. Although all backend functionality is deployed as a single Spring Boot application, the internal structure follows strict module boundaries to minimize coupling and maximize maintainability.

At the highest level, the application consists of three independent components:

Angular 22 SPA

|

REST API (HTTPS)

|

Spring Boot Backend

|

PostgreSQL

Each component has a single responsibility and communicates only with its adjacent layer.

The Angular application is responsible exclusively for user interaction and presentation logic.

The Spring Boot backend encapsulates all business logic, security, validation and persistence.

PostgreSQL serves as the single source of truth for all persistent application data.

This separation allows each layer to evolve independently while maintaining clearly defined interfaces between them.

3.2 High-Level Component Responsibilities

Angular Application

The frontend is responsible for presenting information to the user and collecting user input.

Its responsibilities include:

- user interface rendering
- routing
- local UI state
- HTTP communication
- form validation
- game interaction
- authentication token handling
- displaying statistics

The frontend must never contain business rules that determine whether an answer is correct.

All business validation is delegated to the backend.

Spring Boot Backend

The backend represents the core of the system.

Its responsibilities include:

- authentication
- authorization
- business logic
- answer validation
- dictionary import
- Excel parsing
- game lifecycle management
- statistics calculation
- persistence
- REST API

The backend owns all business rules.

Whenever a business rule changes, the modification should occur only inside the backend.

PostgreSQL Database

The database stores all persistent information.

It contains:

- users
- dictionaries
- cards
- games
- answers
- statistics
- audit information

The database never contains presentation logic.

Derived values are only stored when there is a measurable performance benefit.

3.3 Layered Architecture

The backend follows a classical layered architecture.

Presentation Layer

|

Application Layer

|

Domain Layer

|

Persistence Layer

|

Database

Each layer communicates only with the layer directly below it.

This prevents tight coupling between unrelated parts of the application.

Presentation Layer

Responsible for REST endpoints.

Responsibilities include:

- request validation
- authentication
- authorization entry point
- request mapping
- response mapping
- HTTP status codes

Controllers should remain thin and contain no business logic.

Application Layer

Contains use cases and orchestration logic.

Examples include:

- creating a game
- importing a dictionary
- validating an answer
- finishing a game
- calculating statistics

Application services coordinate domain services and repositories.

Domain Layer

Contains the business rules of the application.

Examples include:

- answer validation
- card selection algorithms
- statistics aggregation
- dictionary consistency rules

The domain layer should remain independent of HTTP and database concerns whenever possible.

Persistence Layer

Responsible for database communication.

Responsibilities include:

- entity persistence
- database queries
- transactions
- optimistic locking
- custom SQL where necessary

Repositories should not contain business logic.

3.4 Modular Structure

The backend is organized using a feature-oriented package structure.

`com.deutschlingodeck`

|— authentication

|— user

|— dictionary

|— answer

|— game

|— statistics

|— common

└— infrastructure

Each module owns its own controllers, services, repositories, entities and DTOs.

This approach minimizes dependencies between business domains and makes future extraction into independent services significantly easier if required.

3.5 Module Responsibilities

Authentication Module

Responsible for:

- registration
- login
- JWT generation
- refresh tokens
- password management

User Module

Responsible for:

- user profile
- user preferences
- account settings

Dictionary Module

Responsible for:

- dictionary upload
- Excel parsing
- dictionary management
- card management
- dictionary validation

Game Module

Responsible for:

- creating games
- card sequencing
- game lifecycle
- game persistence
- game completion

Answer Module

Responsible for:

- answer normalization
- answer comparison
- validation
- accepted answers
- validation result generation

This module is intentionally separated from the Game module because answer validation is expected to become significantly more sophisticated over time.

Future AI-assisted validation should be implemented inside this module without affecting the rest of the application.

Statistics Module

Responsible for:

- updating aggregated statistics
- retrieving statistics
- dashboard calculations
- future spaced repetition algorithms

Common Module

Contains functionality shared across multiple modules.

Examples include:

- exception handling
- shared DTOs
- utility classes
- configuration
- security helpers
- constants

Infrastructure Module

Contains technology-specific implementations.

Examples include:

- Flyway configuration
- Docker-related configuration
- external libraries
- persistence adapters
- monitoring integration

3.6 Communication Principles

Communication inside the application follows strict dependency rules.

Frontend communicates only with REST endpoints.

Controllers communicate only with application services.

Application services communicate with domain services.

Application services communicate with repositories.

Repositories communicate only with PostgreSQL.

Direct communication between unrelated modules should be avoided.

Whenever one module requires functionality from another module, it should communicate through well-defined service interfaces rather than directly accessing repositories.

This keeps module boundaries explicit and prevents hidden dependencies.

3.7 Transaction Boundaries

Every business operation should execute inside a clearly defined transaction.

Examples include:

- dictionary import
- answer submission
- game completion
- statistics updates

Each transaction should either complete successfully or roll back entirely.

Partial persistence of business operations is not acceptable.

For example, an answer must never be stored without updating the corresponding statistics.

Likewise, a dictionary must never be partially imported.

3.8 Source of Truth

One of the most important architectural principles is defining ownership of data.

The backend is the only source of truth for business data.

The frontend maintains only temporary UI state.

The database remains the authoritative source for persistent information.

Historical answers represent immutable events.

Aggregated statistics are derived from those events and may always be reconstructed if necessary.

This event-first approach provides excellent auditability while maintaining high performance through aggregated statistics.

3.9 Scalability Considerations

Although DeutschLingoDeck is implemented as a modular monolith, the architecture intentionally avoids decisions that would prevent future scaling.

Modules communicate through service interfaces rather than direct implementation dependencies.

Business logic is isolated from transport protocols.

Database access is encapsulated behind repositories.

Authentication is stateless.

Because of these decisions, individual modules could later be extracted into independent services if business requirements justify the additional operational complexity.

No architectural decision made in the initial version should prevent future evolution of the system.

3.10 Architectural Summary

The overall architecture emphasizes simplicity, maintainability and clear ownership of responsibilities.

Rather than introducing unnecessary complexity through distributed services or excessive abstraction, the system focuses on well-defined module boundaries, strong separation of concerns and transactional consistency.

This architecture provides a stable foundation for the first production release while remaining flexible enough to support future capabilities such as AI-assisted answer validation, adaptive learning algorithms, advanced analytics and additional client applications without requiring fundamental architectural redesign.

4. Architecture Decision Records (ADR)

4.1 Purpose

Every software system is the result of numerous architectural decisions. While implementation details may evolve over time, architectural decisions have a significantly longer lifespan and directly influence maintainability, scalability and operational complexity.

To ensure long-term consistency, the most important decisions made during the design of DeutschLingoDeck are documented using the Architecture Decision Record (ADR) approach.

Each ADR describes:

- the problem being solved,
- the considered alternatives,
- the chosen solution,
- the rationale behind the decision,
- the long-term consequences.

This allows future developers to understand not only *what* was chosen, but *why* the decision was made.

ADR-001 - Modular Monolith Instead of Microservices

Status

Accepted

Context

The application consists of several closely related business domains:

- Users
- Dictionaries
- Cards
- Games
- Answers
- Statistics

These domains share the same transactional boundaries and operate on the same relational data model.

Most business operations require updating multiple entities within a single transaction.

For example:

- storing an answer
- updating card statistics
- updating game statistics

must either all succeed or all fail.

Splitting these operations across multiple services would require distributed transactions or asynchronous consistency mechanisms.

Considered Alternatives

Alternative 1 — Microservices

Advantages:

- independent deployments
- isolated scalability
- technology flexibility

Disadvantages:

- distributed transactions
- service discovery
- network latency
- API versioning
- operational overhead
- significantly higher development complexity

Alternative 2 — Modular Monolith

Advantages:

- single deployment
- ACID transactions
- simpler debugging
- lower operational cost
- easier local development
- clear module boundaries

Disadvantages:

- single deployment unit
- requires discipline to maintain module boundaries

Decision

The system will be implemented as a modular monolith.

Each business capability will be implemented as an independent feature module inside one Spring Boot application.

Consequences

The architecture remains simple while preserving transactional consistency.

Future migration to microservices remains possible because feature boundaries are explicitly defined.

ADR-002 - Spring Boot as Backend Framework

Status

Accepted

Context

The backend requires:

- REST APIs
- authentication
- authorization
- validation
- database access
- transaction management
- scheduled tasks
- production readiness

Considered Alternatives

Spring Boot

Micronaut

Quarkus

Node.js

ASP.NET

Decision

Spring Boot has been selected.

Rationale

Spring Boot provides a mature ecosystem with first-class support for:

- Spring Security
- Spring Data JPA
- Bean Validation
- Flyway
- Actuator
- Testing
- Dependency Injection

It is one of the most widely adopted enterprise frameworks and offers excellent long-term maintainability.

Consequences

The project benefits from a mature ecosystem, extensive documentation and a large developer community.

ADR-003 - PostgreSQL as Primary Database

Status

Accepted

Context

The application stores strongly related relational data with transactional consistency requirements.

Complex joins, foreign keys and aggregations are essential parts of the domain model.

Considered Alternatives

PostgreSQL

MySQL

MongoDB

SQLite

Decision

PostgreSQL will be used as the primary database.

Rationale

PostgreSQL provides:

- excellent ACID compliance
- advanced indexing
- JSON support
- Common Table Expressions
- UPSERT operations
- Row-Level Security
- mature tooling

It is particularly well suited for applications combining transactional workloads with analytical queries.

Consequences

The application benefits from strong consistency while remaining capable of supporting future analytical features.

ADR-004 - Angular as Frontend Framework

Status

Accepted

Context

The frontend requires:

- component-based architecture
- routing
- forms
- reactive programming
- state management
- enterprise maintainability

Considered Alternatives

Angular

React

Vue

Svelte

Decision

Angular has been selected.

Rationale

Angular provides a complete application framework rather than only a UI library.

Features such as dependency injection, routing, HTTP communication, reactive forms and standalone components are integrated into the framework.

This reduces architectural fragmentation and simplifies long-term maintenance.

Consequences

The project follows Angular best practices and benefits from a strongly opinionated architecture.

ADR-005 - Angular Signals Instead of Global State Management

Status

Accepted

Context

The application requires reactive UI state.

However, the majority of application state is transient and scoped to a single feature.

Considered Alternatives

Angular Signals

NgRx

Akita

NGXS

Decision

Angular Signals will be used for local application state.

Rationale

Most state belongs to individual pages or active game sessions.

Introducing a global Redux-style store would unnecessarily increase complexity.

Signals provide a lightweight reactive programming model while remaining fully integrated into Angular.

Consequences

The application remains easier to understand while preserving the option of introducing NgRx in the future if business complexity increases.

ADR-006 - REST Instead of GraphQL

Status

Accepted

Context

The frontend communicates exclusively with a single backend.

Data structures are well known and stable.

Considered Alternatives

REST

GraphQL

gRPC

Decision

REST APIs will be used.

Rationale

REST offers:

- simplicity
- mature tooling
- predictable caching
- straightforward documentation
- easy integration with Angular HttpClient

GraphQL would introduce additional complexity without solving any actual business problem.

Consequences

The API remains simple, predictable and easy to maintain.

ADR-007 - JWT-Based Authentication

Status

Accepted

Context

The application is implemented as a Single Page Application.

The backend should remain stateless.

Decision

Authentication will be based on:

- JWT Access Token
- Refresh Token

Rationale

JWT authentication:

- removes server-side sessions
- enables horizontal scalability
- simplifies deployment
- integrates naturally with REST APIs

Consequences

Authentication remains independent of backend instances.

ADR-008 - Event History as the Source of Truth

Status

Accepted

Context

The application continuously calculates learning statistics.

Statistics may evolve as new algorithms are introduced.

Decision

Every submitted answer will be stored permanently.

Statistics will always be derived from historical answers.

Rationale

Historical answers represent immutable business events.

Aggregated statistics can always be recalculated.

This approach prevents permanent data loss if aggregation logic changes.

Consequences

Historical data remains available for:

- statistics
- analytics
- AI evaluation
- spaced repetition
- debugging

ADR-009 - Aggregated Statistics

Status

Accepted

Context

Calculating statistics from millions of answers for every dashboard request would eventually become expensive.

Decision

Maintain aggregated statistics alongside historical answers.

Rationale

The application stores:

- immutable event history
- aggregated counters

This combines correctness with performance.

Consequences

Statistics become extremely fast while remaining fully reconstructable.

ADR-010 - Feature-Based Project Structure

Status

Accepted

Context

Large enterprise applications become difficult to maintain when organized exclusively by technical layers.

Decision

Both frontend and backend will follow a feature-oriented structure.

Example backend modules include:

- authentication
- user
- dictionary
- answer
- game
- statistics

Rationale

Keeping controllers, services, DTOs, entities and repositories close to their business domain improves maintainability and reduces cognitive load.

Consequences

Future developers can understand individual business domains without navigating the entire project.

ADR Summary

The architectural decisions documented in this chapter define the long-term technical direction of the project.

These decisions intentionally prioritize simplicity, maintainability and production readiness over unnecessary architectural complexity.

Future implementation work should follow these decisions consistently unless a new ADR explicitly supersedes an existing one.

Documenting architectural decisions in this form creates a shared understanding among all contributors and significantly reduces the likelihood of inconsistent implementation choices during the lifetime of the project.

5. Technology Stack

5.1 Overview

The technology stack has been selected with three primary objectives:

- long-term maintainability,
- production readiness,
- simplicity without sacrificing scalability.

Rather than choosing technologies solely based on popularity, every component has been selected because it naturally fits the architectural goals of the system.

The application follows a modern full-stack architecture consisting of four logical layers:

- Frontend
- Backend
- Database
- Infrastructure

Each layer uses technologies that complement one another while minimizing unnecessary complexity.

5.2 Technology Overview

Layer	Technology
Frontend	Angular 22
Language	TypeScript
Styling	SCSS
Backend	Java 21 LTS

Framework	Spring Boot 3.x
Security	Spring Security
ORM	Spring Data JPA + Hibernate
Database	PostgreSQL 16
Database Migration	Flyway
Excel Import	Apache POI
Authentication	JWT (Access + Refresh Token)
Build Tool	Maven
Containerization	Docker
Orchestration	Docker Compose
Reverse Proxy	Nginx
Version Control	Git
Testing	JUnit, Mockito, Testcontainers

API Testing	Postman
IDE	IntelliJ IDEA / Visual Studio Code

5.3 Frontend Technologies

Angular 22

Angular is the foundation of the frontend application.

The project uses the latest standalone architecture introduced in modern Angular versions.

The application intentionally avoids legacy concepts such as NgModules.

The frontend architecture is based on:

- Standalone Components
- Angular Signals
- Dependency Injection
- Lazy Loading
- Functional Route Guards
- Functional Interceptors
- Modern Control Flow (@if, @for, @switch)

Angular was selected because it provides a complete enterprise framework rather than only a UI rendering library.

Its opinionated architecture improves long-term maintainability by encouraging consistent development practices across the entire application.

TypeScript

All frontend code is written in TypeScript.

Using TypeScript instead of JavaScript provides:

- static type checking
- better IDE support
- safer refactoring

- improved code readability
- earlier detection of programming errors

Strong typing is especially valuable in applications with extensive REST communication and complex domain models.

Angular Signals

Angular Signals are used as the primary reactive programming model.

Signals manage local application state such as:

- active game session
- selected dictionary
- current flashcard
- progress indicators
- UI state

Signals replace much of the reactive complexity traditionally handled by RxJS Subjects or global state libraries.

Business data retrieved from the backend remains immutable and originates exclusively from REST APIs.

RxJS

RxJS is still used where asynchronous streams are the natural abstraction.

Examples include:

- HTTP requests
- route changes
- authentication events
- retry mechanisms
- debouncing
- polling

Signals and RxJS complement each other rather than compete.

Signals manage state.

RxJS manages asynchronous event streams.

SCSS

The application uses SCSS for styling.

SCSS provides:

- variables
- nesting
- reusable mixins
- better organization
- modular styles

Component-level styles remain isolated by Angular's style encapsulation.

5.4 Backend Technologies

Java 21 LTS

The backend is implemented using Java 21 Long-Term Support.

Reasons for choosing Java include:

- stability
- maturity
- excellent tooling
- strong ecosystem
- enterprise adoption

Java 21 also introduces Virtual Threads, allowing highly efficient handling of blocking I/O operations such as Excel parsing and database communication.

Spring Boot

Spring Boot provides the foundation of the backend.

It supplies:

- dependency injection
- REST framework
- configuration management
- validation
- transaction management
- application lifecycle
- production tooling

Spring Boot significantly reduces boilerplate code while promoting well-established architectural practices.

Spring Security

Spring Security provides authentication and authorization.

Responsibilities include:

- login
- JWT validation
- endpoint protection
- password hashing
- authorization rules

Security is implemented centrally rather than individually inside controllers.

Spring Data JPA

Persistence is implemented using Spring Data JPA.

Advantages include:

- repository abstraction
- pagination
- derived queries
- transaction integration
- reduced boilerplate

Native SQL is introduced only where PostgreSQL-specific features provide measurable benefits.

Examples include:

- UPSERT operations
- advanced statistics updates

Hibernate

Hibernate serves as the JPA implementation.

Responsibilities include:

- object-relational mapping
- lazy loading
- dirty checking
- SQL generation
- transaction synchronization

Entities remain persistence models and are never exposed through REST APIs.

Bean Validation

Request validation uses Jakarta Bean Validation.

Validation annotations include:

- `@NotNull`
- `@NotBlank`
- `@Size`
- `@Email`
- custom validators

Validation occurs before business logic is executed.

5.5 Database Technologies

PostgreSQL

PostgreSQL serves as the primary database.

It provides:

- ACID compliance
- foreign keys
- transactions
- indexing
- JSON support
- UPSERT
- Common Table Expressions
- Row-Level Security

The database has been selected for its robustness and excellent support for both transactional and analytical workloads.

Flyway

Database schema changes are managed using Flyway.

Every structural modification is introduced through versioned migration scripts.

Example:

V1__Create_users.sql

V2__Create_dictionary_tables.sql

V3__Create_games.sql

V4__Create_statistics.sql

Using migrations guarantees reproducible database deployments across all environments.

Direct schema modifications are never performed manually.

5.6 Excel Processing

Apache POI

Dictionary import relies on Apache POI.

Responsibilities include:

- reading XLSX files
- validating workbook structure
- extracting rows
- detecting formatting issues
- streaming large files

Apache POI is the industry standard library for Microsoft Office document processing.

5.7 Authentication Technologies

Authentication is based on JSON Web Tokens.

The authentication flow consists of:

- Access Token
- Refresh Token

Access tokens remain short-lived.

Refresh tokens allow obtaining new access tokens without forcing users to log in repeatedly.

Passwords are hashed using BCrypt before storage.

No plaintext passwords are ever persisted.

5.8 Infrastructure Technologies

Docker

Every application component is containerized.

Containerization guarantees:

- consistent environments
- reproducible deployments
- simplified onboarding
- isolation

Each service runs independently.

Docker Compose

Docker Compose orchestrates:

- Angular frontend
- Spring Boot backend
- PostgreSQL database

Networking between services is handled automatically.

Environment-specific configuration is supplied through environment variables.

Nginx

The Angular application is served through Nginx.

Nginx provides:

- static file hosting
- reverse proxy
- compression
- caching
- HTTPS termination
- security headers

This architecture closely resembles a production deployment.

5.9 Development Tools

Git

Git serves as the version control system.

Recommended branching strategy:

- main
- develop
- feature/*
- release/*
- hotfix/*

Feature branches should remain short-lived.

Pull Requests should undergo code review before merging.

Gradle

Gradle manages:

- dependencies
- builds
- plugins
- packaging

The project follows the standard Gradle directory structure.

IntelliJ IDEA

Primary backend development environment.

Provides:

- advanced refactoring
- Spring integration
- debugging
- profiling
- database tools

Visual Studio Code

Recommended frontend development environment.

Provides:

- Angular Language Service
- TypeScript support
- Docker integration
- Git integration

Postman

Used for:

- REST API testing
- authentication testing
- endpoint verification
- regression testing

Collections should be version-controlled together with the application.

5.10 Testing Technologies

The project follows a layered testing strategy.

Backend

- JUnit 5
- Mockito
- Spring Boot Test
- Testcontainers

Testcontainers ensure that integration tests execute against a real PostgreSQL instance rather than an in-memory database.

Frontend

Angular testing stack includes:

- Jasmine
- Karma (or future migration to Vitest)
- Angular Testing Library

Testing focuses on:

- components
- services
- routing
- state management

5.11 Future Technology Roadmap

The current technology stack intentionally leaves room for future additions.

Potential future technologies include:

AI

- OpenAI API
- Anthropic Claude API
- Local LLMs through Ollama

Observability

- Prometheus

- Grafana
- Loki

Caching

- Redis

Search

- Elasticsearch

Messaging

- RabbitMQ
- Apache Kafka

Deployment

- Kubernetes
- Helm

Monitoring

- OpenTelemetry

These technologies are intentionally excluded from the initial implementation because they would introduce unnecessary complexity before solving a concrete business problem.

5.12 Technology Stack Summary

The selected technology stack emphasizes maturity, stability and long-term maintainability over short-term trends.

Every chosen technology integrates naturally with the surrounding ecosystem and contributes to a cohesive architecture.

The combination of Angular, Spring Boot and PostgreSQL provides a proven enterprise-grade foundation capable of supporting both the initial implementation and future evolution of the platform without requiring significant architectural changes.

6. Domain Model

6.1 Overview

The Domain Model defines the core business concepts of DeutschLingoDeck and the relationships between them.

Unlike the database schema, which focuses on persistence, the domain model describes the business language used throughout the application.

Every feature implemented by the system revolves around these domain entities. The purpose of this chapter is to establish a shared vocabulary that is understood consistently across the frontend, backend and database.

The domain model intentionally remains independent of implementation details such as REST APIs, database tables or UI components.

6.2 Domain Overview

The application consists of seven primary business entities.

User

|

├─ Dictionary

| |

| └─ Card

|

├─ Game

| |

| └─ Answer

|

└─ Statistics

Each entity represents a business concept rather than a database table.

Some entities eventually become database tables, while others may exist only as aggregates or service-level abstractions.

6.3 User

Purpose

A User represents an authenticated person using the application.

Every persistent resource belongs to exactly one user.

The user is the highest-level ownership boundary within the system.

No business object may exist without an owner.

Responsibilities

A user can:

- register an account
- authenticate
- upload dictionaries
- manage dictionaries
- start games
- answer cards
- review previous games
- view statistics
- update profile information

Business Rules

A user may own multiple dictionaries.

A user may play an unlimited number of games.

A user cannot access another user's resources.

Deleting a user should never happen as a physical delete in normal operation.

Instead, accounts should be deactivated to preserve historical learning data.

6.4 Dictionary

Purpose

A Dictionary represents a logical collection of learning cards.

It is the primary source of learning content.

A dictionary belongs to exactly one user.

Responsibilities

A dictionary stores:

- metadata
- language information
- cards
- import history

A dictionary itself contains no statistics.

Statistics are maintained separately.

Business Rules

Each dictionary belongs to one user.

Dictionary names must be unique per user.

Different users may have dictionaries with identical names.

Deleting a dictionary must not remove historical games.

The dictionary should instead be marked as inactive.

Metadata

Typical dictionary metadata includes:

- name
- description
- source language
- target language

- number of cards
- creation date
- last modification date

Future versions may also include:

- tags
- visibility
- sharing permissions
- version information

6.5 Card

Purpose

A Card represents the smallest learning unit.

Each card belongs to exactly one dictionary.

Cards are immutable learning objects.

Learning progress is never stored inside the card itself.

Card Types

The system supports multiple card categories.

Examples include:

- Word
- Noun
- Verb
- Phrase
- Sentence

Different card types may require different validation strategies.

Card Data

Each card contains:

- source text

- primary translation
- optional accepted translations
- article (if applicable)
- example sentence
- notes
- difficulty level
- position inside the imported dictionary

Business Rules

Cards cannot exist without a dictionary.

Cards may have multiple accepted answers.

Cards never store user-specific information.

6.6 Game

Purpose

A Game represents one learning session.

Games are created whenever a user starts practicing with a selected dictionary.

Games are independent.

Multiple games may be created from the same dictionary.

Lifecycle

The lifecycle of a game consists of:

Created



Started



In Progress



Finished or Abandoned

Each game stores its own metadata.

Game Metadata

Typical metadata includes:

- start time
- finish time
- duration
- selected dictionary
- number of cards
- completion status

Business Rules

A game belongs to exactly one user.

A game references exactly one dictionary.

Every answer submitted during a game becomes part of that game's history.

Finished games cannot be modified.

6.7 Answer

Purpose

An Answer represents one interaction between the user and a card.

Answers form the immutable event history of the application.

They are the foundation for all future statistics.

Stored Information

Each answer stores:

- card reference
- game reference
- user response
- expected response
- normalized response
- validation result
- correctness
- response time
- timestamp

Business Rules

Answers cannot be modified.

Answers cannot be deleted.

Every submitted answer remains permanently available for auditing and analytics.

Validation Result

Validation may produce several outcomes.

Examples include:

- Correct
- Wrong Article
- Wrong Translation
- Missing Word

- Extra Word
- Typographical Error
- Partially Correct

This model allows significantly richer feedback than a simple Boolean value.

6.8 Statistics

Purpose

Statistics represent aggregated information derived from historical answers.

Statistics are not the primary source of truth.

Instead, they are optimized read models designed for fast retrieval.

Levels of Aggregation

The application maintains statistics on several levels.

Card Statistics

Stores learning progress for an individual card.

Examples:

- times shown
- correct answers
- incorrect answers
- success rate
- last appearance
- last correct answer

Dictionary Statistics

Summarizes learning progress across all cards inside a dictionary.

Examples:

- completed games
- average score
- mastered cards

- difficult cards

User Statistics

Provides a global overview.

Examples:

- total games
- total answers
- total study time
- overall success rate
- learning streak
- favorite dictionaries

Business Rules

Statistics may always be recalculated from historical answers.

No statistical value should become the only source of truth.

6.9 Import

Although not persisted as a long-lived business entity, dictionary import is an important domain concept.

The import process consists of several business stages:

Upload

↓

Validation

↓

Parsing

↓

Business Validation

↓

Persistence



Import Report

The import process is atomic.

Either the complete dictionary is imported successfully or nothing is stored.

6.10 Ownership Model

Ownership is one of the fundamental concepts of the application.

Every persistent object belongs to exactly one user.

Ownership is inherited through the domain hierarchy:

User



Dictionary



Card



Game



Answer



Statistics

Every authorization decision is ultimately based on this ownership chain.

No resource may exist without a valid owner.

6.11 Domain Invariants

The following business rules must always remain true.

A card always belongs to exactly one dictionary.

A dictionary always belongs to exactly one user.

A game always references one dictionary.

An answer always references one game.

Statistics always belong to one user.

Historical answers are immutable.

Statistics are derived data.

User ownership can never be bypassed.

These rules are considered domain invariants and should be enforced throughout the application.

6.12 Aggregate Boundaries

The application naturally divides into several aggregates.

User Aggregate

Root:

User

Contains:

- profile
- credentials
- preferences

Dictionary Aggregate

Root:

Dictionary

Contains:

- cards
- import information

Game Aggregate

Root:

Game

Contains:

- answers
- session metadata

Statistics Aggregate

Root:

Statistics

Contains:

- card statistics
- dictionary statistics
- user statistics

Keeping aggregates independent minimizes transactional complexity while maintaining strong consistency within each business boundary.

6.13 Domain Events (Future Extension)

Although the first version updates statistics synchronously, the domain model is intentionally designed around business events.

Potential future events include:

- DictionaryImported
- GameStarted
- CardDisplayed
- AnswerSubmitted
- AnswerValidated
- GameFinished
- StatisticsUpdated

These events could later be published to an event bus without requiring significant changes to the business model.

This makes the architecture naturally extensible toward event-driven processing while preserving the simplicity of the current modular monolith.

6.14 Domain Model Summary

The domain model establishes a clear separation between business concepts and technical implementation.

Each entity has a single well-defined responsibility and participates in a structured ownership hierarchy.

Historical answers act as immutable business events, while statistics provide optimized read models derived from those events.

This approach ensures consistency, auditability and future extensibility while keeping the business model simple, expressive and independent of implementation details.

The following chapter builds directly upon this domain model and translates it into a relational PostgreSQL database schema.

7. Database Design

7.1 Overview

The PostgreSQL database serves as the primary persistence layer of DeutschLingoDeck and represents the single source of truth for all business data.

The database is designed according to the following principles:

- strong data consistency
- normalized relational model
- explicit ownership
- transactional integrity
- auditability
- future extensibility

The schema intentionally separates immutable historical data from aggregated statistics. Every user interaction is permanently stored, while derived values are maintained separately to optimize query performance.

7.2 Database Design Principles

The following principles guide every database design decision.

Normalization

Business entities are normalized to eliminate unnecessary duplication while preserving query efficiency.

Only carefully selected denormalized values are stored when they provide measurable performance benefits.

Examples include:

- total number of cards in a dictionary
- game score
- aggregated statistics

Referential Integrity

Every relationship is protected by foreign key constraints.

Business entities cannot exist without valid parent entities.

Examples:

- cards require dictionaries
- games require users
- answers require games
- statistics require users

Immutable Historical Data

Historical records are never modified after creation.

Examples include:

- submitted answers
- completed games
- import logs

Historical data represents business events and serves as the foundation for future analytics.

Soft Deletion

Business objects that may contain historical references are never physically deleted.

Instead, soft deletion is implemented through a nullable timestamp.

`deleted_at TIMESTAMPTZ NULL`

This preserves historical integrity while hiding inactive objects from normal application workflows.

Derived Data

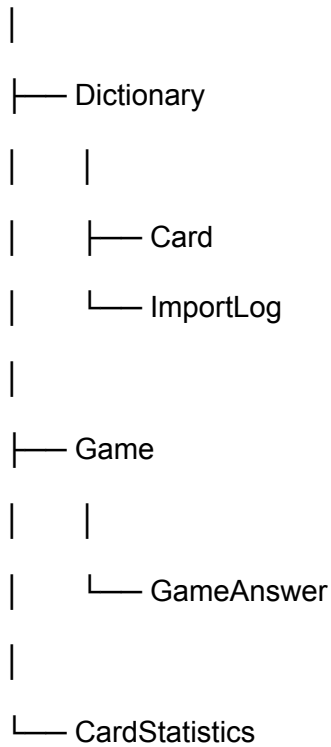
Statistics are stored as optimized read models.

Their values are derived from immutable historical answers and may always be reconstructed.

7.3 Entity Relationship Overview

The domain model translates into the following relational structure.

User



Each relationship follows strict ownership rules.

No entity may exist without its parent.

7.4 User Table

Purpose

Stores application users.

This table represents the ownership root of the entire database.

Every persistent resource ultimately belongs to a user.

Main Attributes

- id
- email
- password_hash
- display_name
- created_at
- updated_at
- deleted_at

Business Rules

Email addresses must be unique.

Passwords are stored exclusively as BCrypt hashes.

Users are never physically deleted.

Authentication credentials are isolated from application data.

Indexes

Unique index:

email

7.5 Dictionary Table

Purpose

Represents one imported dictionary.

Each dictionary belongs to exactly one user.

Main Attributes

- id
- user_id
- name
- description

- source_language
- target_language
- card_count
- created_at
- updated_at
- deleted_at

Business Rules

Dictionary names must be unique per user.

The same name may exist for different users.

Soft deletion preserves historical games.

Indexes

Composite unique index:

(user_id, name)

Additional index:

user_id

7.6 Card Table

Purpose

Represents one learning card.

Cards are immutable learning objects.

They contain no user-specific data.

Main Attributes

- id
- dictionary_id
- card_type
- article
- source_text
- primary_translation
- example
- notes
- difficulty_level
- position
- active
- created_at
- updated_at
- deleted_at

Business Rules

Cards belong to one dictionary.

Cards may have multiple accepted answers.

Cards never contain learning statistics.

Indexes

dictionary_id

(dictionary_id, position)

7.7 Accepted Answers Table

Purpose

Stores alternative valid answers for one card.

This allows multiple correct translations without duplicating cards.

Main Attributes

- id
- card_id
- answer
- is_primary

Business Rules

Every accepted answer belongs to exactly one card.

One answer may be designated as the primary answer.

The validation engine compares user input against all accepted answers.

7.8 Game Table

Purpose

Represents one learning session.

Main Attributes

- id
- user_id
- dictionary_id
- status
- started_at
- finished_at
- duration_seconds
- total_cards
- correct_answers
- incorrect_answers

Business Rules

A game always belongs to one user.

A game always references one dictionary.

Finished games become immutable.

Status Values

Possible values include:

- CREATED
- STARTED
- FINISHED
- ABANDONED

Indexes

user_id

(user_id, started_at)

7.9 Game Answers Table

Purpose

Stores every answer submitted during every game.

This table represents the historical event log of the application.

Main Attributes

- id
- game_id
- card_id
- given_answer
- expected_answer
- normalized_answer
- validation_result
- is_correct
- response_time_ms

- answered_at

Business Rules

Answers are immutable.

Answers cannot be deleted.

Answers cannot be modified.

Every submitted answer becomes part of the permanent learning history.

Indexes

game_id

card_id

7.10 Card Statistics Table

Purpose

Stores aggregated learning statistics for one user and one card.

Main Attributes

- user_id
- dictionary_id
- card_id
- times_shown
- times_correct
- times_incorrect
- last_shown_at
- last_correct_at

Business Rules

Statistics never replace historical answers.

Statistics are updated transactionally.

Statistics may always be reconstructed.

Unique Constraint

(user_id,
dictionary_id,
card_id)

Indexes

user_id

(user_id, dictionary_id)

7.11 Import Log Table

Purpose

Stores information about every dictionary import.

The original Excel file is not preserved.

Only metadata is stored.

Main Attributes

- user_id

- dictionary_id
- filename
- import_format_version
- imported_rows
- failed_rows
- status
- error_report
- created_at

Business Rules

Import logs provide auditability.

They are independent from the dictionary itself.

Failed imports remain visible for troubleshooting.

7.12 Ownership Model

Ownership is enforced through foreign keys.

The ownership chain is:

User

↓

Dictionary

↓

Card

↓

Game

↓

Answer

↓

Statistics

Every authorization decision ultimately resolves ownership through this hierarchy.

7.13 Transaction Strategy

Database transactions follow business use cases.

Typical transactional operations include:

Dictionary Import

- create dictionary
- create cards
- update card count
- create import log

Either the entire operation succeeds or it is rolled back.

Answer Submission

- store answer
- update statistics
- update game counters

All operations execute inside a single transaction.

Game Completion

- update finish time
- calculate duration
- finalize score
- update status

7.14 Optimistic Locking

Future concurrent modifications should use optimistic locking.

Entities that are expected to change over time should include:

@Version

private Long version;

This prevents lost updates while avoiding unnecessary pessimistic locking.

Optimistic locking is particularly useful for:

- user profile updates
- dictionary editing
- settings

Historical entities such as answers do not require versioning because they are immutable.

7.15 Indexing Strategy

Indexes are created based on expected query patterns rather than automatically for every column.

Primary lookup scenarios include:

User login

email

User dictionaries

user_id

Cards inside dictionary

dictionary_id

User games

(user_id, started_at)

Card statistics

(user_id, dictionary_id)

Indexes should be periodically reviewed as production usage patterns evolve.

7.16 Database Migrations

Database schema changes are managed exclusively through Flyway.

Every structural modification is introduced as a versioned migration.

Example:

V1__Users.sql

V2__Dictionary.sql

V3__Cards.sql

V4__Games.sql

V5__Answers.sql

V6__Statistics.sql

Database changes are never applied manually.

Each migration must be:

- deterministic
- repeatable
- reversible whenever practical

7.17 Future Database Extensions

The schema intentionally leaves room for future features.

Potential additions include:

Learning Algorithms

- spaced repetition data
- review intervals
- ease factors

AI Features

- AI evaluation history
- explanation cache
- suggested corrections

Voice Learning

- pronunciation recordings
- speech recognition scores

Analytics

- learning streaks
- heat maps
- daily activity
- progress trends

These additions can be implemented without redesigning the existing schema.

7.18 Database Design Summary

The database design prioritizes consistency, auditability and maintainability.

Historical answers remain immutable and serve as the authoritative source of learning history.

Aggregated statistics provide efficient read performance while remaining fully reconstructable from historical events.

Ownership is enforced throughout the schema using explicit foreign key relationships, ensuring complete isolation between users.

The resulting data model is optimized for transactional workloads while remaining flexible enough to support future analytical and AI-driven features without fundamental structural changes.

8. Backend Architecture

8.1 Overview

The backend represents the core of the DeutschLingoDeck system.

While the frontend is responsible for presentation and user interaction, all business rules, validation logic, authorization, persistence and transactional consistency are implemented inside the backend.

The backend follows a modular monolithic architecture built with Spring Boot. Each business capability is implemented as an independent feature module with clearly defined responsibilities and minimal coupling to other modules.

The primary architectural goals are:

- maintainability
- readability
- modularity
- transactional consistency
- testability
- production readiness

The backend is designed to support future growth without requiring a fundamental architectural redesign.

8.2 Architectural Layers

The backend follows a layered architecture.

REST API

|

Controllers

|

Application Services

|

Domain Services

|

Repositories

|
PostgreSQL

Each layer has a clearly defined responsibility.

Dependencies always point downward.

Lower layers must never depend on higher layers.

8.3 Layer Responsibilities

Controller Layer

Controllers expose REST endpoints.

Responsibilities include:

- request mapping
- request validation
- authentication entry point
- authorization entry point
- HTTP response generation
- status codes

Controllers should remain extremely thin.

They should never contain business logic.

A controller should typically consist only of:

- request validation
- service invocation
- response mapping

Application Service Layer

Application Services orchestrate business use cases.

Examples include:

- importing a dictionary
- starting a game

- submitting an answer
- finishing a game
- retrieving statistics

Application Services coordinate multiple domain services and repositories while defining transaction boundaries.

Business workflows belong here.

Domain Service Layer

Domain Services implement business rules.

Examples include:

- answer validation
- card selection
- score calculation
- statistics aggregation
- dictionary validation

Domain Services should not know anything about:

- HTTP
- REST
- JSON
- Controllers

Their only responsibility is business logic.

Repository Layer

Repositories encapsulate persistence.

Responsibilities include:

- CRUD operations
- custom queries
- pagination
- filtering
- database access

Repositories never contain business rules.

8.4 Package Structure

The backend follows a feature-oriented package structure.

com.deutschlingodeck

authentication

user

dictionary

answer

game

statistics

common

infrastructure

Each feature contains its own internal layers.

Example:

dictionary

controller

service

domain

repository

entity

dto

mapper

validator

exception

This keeps related code together and greatly improves maintainability.

8.5 Dependency Rules

The project follows strict dependency rules.

Allowed dependencies:

Controller

↓

Application Service

↓

Domain Service

↓

Repository

Forbidden dependencies:

Controller → Repository

Controller → Entity

DTO → Repository

Entity → DTO

Repository → Controller

Cross-module repository access

Every dependency should follow business ownership rather than convenience.

8.6 Controllers

Controllers expose REST APIs.

Naming convention:

AuthenticationController

DictionaryController

GameController

StatisticsController

UserController

Each controller should expose only endpoints belonging to its own feature.

Controllers must never call repositories directly.

8.7 Application Services

Application Services represent complete business use cases.

Examples include:

DictionaryApplicationService

GameApplicationService

StatisticsApplicationService

AuthenticationApplicationService

Responsibilities:

- orchestrating workflows
- transaction management
- coordinating domain services
- coordinating repositories

Application Services answer the question:

"What business process is currently being executed?"

8.8 Domain Services

Domain Services encapsulate business rules.

The application should avoid creating "God Services."

Instead of a single GameService containing thousands of lines of code, responsibilities are distributed.

Example:

GameCreationService

GameCompletionService

CardSelectionService

AnswerValidationService

AnswerNormalizationService

StatisticsAggregationService

DictionaryImportService

ExcelParsingService

Each service performs one clearly defined responsibility.

8.9 DTO Design

The backend never exposes entities directly.

Every API uses DTOs.

DTO categories include:

Request DTOs

Response DTOs

Summary DTOs

Detail DTOs

Projection DTOs

Example:

CreateGameRequest

GameSummaryResponse

DictionaryDetailResponse

CardStatisticsResponse

DTOs are immutable.

Entities remain internal implementation details.

8.10 Entity Design

Entities represent persistence models.

Entities should contain:

- identity
- relationships
- persistence state

Entities should avoid containing infrastructure code.

Business logic that spans multiple entities belongs in Domain Services.

Entity relationships should remain simple and explicit.

8.11 Mapping Strategy

Entity ↔ DTO mapping is isolated inside dedicated mapper classes.

Example:

DictionaryMapper

GameMapper

StatisticsMapper

UserMapper

No controller should perform manual mapping.

Likewise, repositories should never return DTOs directly unless explicitly using projection queries for performance reasons.

8.12 Validation Strategy

Validation occurs on multiple levels.

Level 1

Request validation

Examples:

- mandatory fields
- string length
- email format

Implemented through Bean Validation.

Level 2

Business validation

Examples:

- dictionary ownership
- duplicate dictionary names
- dictionary contains cards
- game belongs to user

Implemented inside Domain Services.

Level 3

Persistence validation

Examples:

- foreign keys
- unique constraints

Handled by PostgreSQL.

This layered validation strategy minimizes unnecessary database interaction while maintaining consistency.

8.13 Exception Handling

The application uses centralized exception handling.

GlobalExceptionHandler converts internal exceptions into standardized HTTP responses.

Typical exception hierarchy:

ApplicationException

BusinessException

ValidationException

AuthenticationException

AuthorizationException

ResourceNotFoundException

ConflictException

Clients never receive stack traces.

All errors follow a consistent JSON structure.

8.14 Transaction Management

Application Services define transaction boundaries.

Typical transactional workflows include:

Dictionary Import

Store Answer

Finish Game

Update Statistics

Each workflow executes inside a single transaction.

Nested transactions should be avoided unless absolutely necessary.

8.15 Logging Strategy

Logging follows structured logging principles.

Every request receives a Correlation ID.

Typical logged information:

- request ID
- authenticated user
- endpoint

- execution time
- validation failures
- unexpected exceptions

Sensitive information such as passwords or JWT tokens must never be logged.

8.16 Security Architecture

Security is centralized.

Authentication is implemented through Spring Security.

Authorization is ownership-based.

Controllers never perform manual authorization checks.

Security responsibilities include:

- JWT validation
- authentication
- endpoint protection
- ownership verification
- password hashing

The security configuration remains isolated from business modules.

8.17 Configuration Management

Configuration is externalized.

Examples include:

Database

JWT

Docker

Logging

CORS

File Upload Limits

Configuration differs between:

Development

Testing

Production

Business logic must never depend on environment-specific configuration.

8.18 Backend Testing Strategy

Testing follows the testing pyramid.

Unit Tests

↓

Integration Tests

↓

API Tests

↓

End-to-End Tests

Repositories are tested against PostgreSQL using Testcontainers.

Application Services are tested independently from controllers.

Controllers verify HTTP behavior.

Business rules are tested at the Domain Service level.

8.19 Future Backend Extensions

The architecture intentionally allows future modules.

Potential additions include:

Notification Module

AI Module

Speech Recognition Module

Recommendation Module

Scheduling Module

Analytics Module

Audit Module

These modules can be introduced without modifying existing feature boundaries.

8.20 Backend Architecture Summary

The backend architecture emphasizes modularity, clear ownership of responsibilities and strict separation of concerns.

Every feature is implemented as an independent module with well-defined boundaries.

Controllers remain thin.

Business logic resides inside dedicated domain services.

Repositories focus exclusively on persistence.

DTOs isolate the API contract from the persistence model.

This architecture minimizes coupling, maximizes testability and provides a stable foundation for long-term evolution while remaining simple enough to support rapid development during the initial phases of the project.

9. Frontend Architecture

9.1 Overview

The frontend is implemented as a modern Single Page Application (SPA) using Angular 22.

Its primary responsibility is to provide a responsive, intuitive and maintainable user interface while delegating all business rules to the backend.

The frontend does not contain business logic such as answer validation, statistics calculation or authorization rules. Instead, it focuses on presentation, user interaction and communication with the backend through REST APIs.

This strict separation keeps the frontend lightweight and ensures that business rules remain centralized inside the backend.

9.2 Architectural Goals

The frontend architecture is designed around the following principles:

- feature-based organization
- reusable UI components
- reactive state management
- lazy loading
- clear separation of concerns
- minimal global state
- strong typing
- production readiness

The architecture should remain understandable even as the application grows.

9.3 Frontend Architecture Overview

The Angular application consists of five logical layers.

Pages

|

Feature Components

|

Reusable UI Components

|

Services

|

REST API

Each layer has a single responsibility.

9.4 Project Structure

The application follows a feature-based organization.

src/app

core/

shared/

features/

layout/

assets/

environments/

Core Module

The Core layer contains singleton services and application-wide functionality.

Examples include:

- authentication
- route guards
- HTTP interceptors
- global configuration
- API configuration
- shared domain models

The Core layer should never depend on feature modules.

Shared Module

The Shared layer contains reusable UI building blocks.

Examples include:

- buttons
- dialogs
- cards
- icons
- loaders
- tables
- form controls

Shared components contain no business logic.

They should be reusable across the entire application.

Feature Modules

Every business capability is implemented as an independent feature.

Examples:

authentication

dashboard

dictionary

game

statistics

profile

settings

Each feature owns:

- pages
- components
- services
- models
- routing
- state

Layout Module

The Layout layer contains components responsible for application structure.

Examples include:

- header
- sidebar
- footer
- navigation
- application shell

Layout components never contain business logic.

9.5 Feature Structure

Each feature follows the same internal organization.

Example:

game/

pages/

components/

services/

models/

state/

guards/

routes.ts

Keeping every feature self-contained significantly improves maintainability.

9.6 Pages vs Components

The architecture distinguishes between Pages and Components.

Pages

Pages represent complete screens.

Examples:

- Login Page
- Dictionary List
- Dictionary Details
- Game Setup
- Active Game
- Game Summary
- Statistics Dashboard

A page coordinates data loading and orchestrates UI components.

Feature Components

Feature Components implement business-specific UI.

Examples:

- Flashcard
- Answer Form
- Progress Bar
- Dictionary Card
- Statistics Chart

These components are reusable only inside their feature.

Shared Components

Shared Components are generic UI building blocks.

Examples:

- Primary Button
- Confirmation Dialog
- Spinner
- Badge
- Modal Window
- Input Field

Shared Components must remain independent from business concepts.

9.7 Routing

The application uses lazy-loaded feature routes.

Example:

/

login

dashboard

dictionaries

game

statistics

profile

settings

Each feature owns its own routing configuration.

The root router only assembles feature routes.

This keeps routing decentralized and improves scalability.

9.8 State Management

The frontend intentionally avoids introducing a global Redux-style architecture.

Most application state is local to individual features.

Examples include:

Game

Selected Dictionary

Current Card

Current Progress

Dialog Visibility

Loading State

Angular Signals provide an excellent solution for these scenarios.

Local State

Each feature owns its own state service.

Example:

GameStateService

DictionaryStateService

StatisticsStateService

These services expose readonly Signals to components.

Only the owning service may mutate state.

Global State

Only truly global information belongs in the Core layer.

Examples include:

Current User

Authentication State

Application Theme

Language

Everything else remains inside the owning feature.

9.9 Signals and RxJS

The project intentionally combines Signals and RxJS.

Signals manage application state.

RxJS manages asynchronous communication.

Examples where Signals are preferred:

- selected dictionary
- active card
- progress

- dialog visibility

Examples where RxJS is preferred:

- HTTP requests
- route changes
- authentication refresh
- retry logic
- polling

Signals should never replace asynchronous streams.

RxJS should never become a global state store.

9.10 Service Layer

Services encapsulate backend communication.

Typical services include:

AuthenticationService

DictionaryService

GameService

StatisticsService

ProfileService

Services remain lightweight.

Responsibilities include:

- REST communication
- request mapping
- response mapping
- error propagation

Business decisions belong to the backend.

9.11 HTTP Communication

The frontend communicates exclusively through REST APIs.

Every request passes through HTTP interceptors.

Interceptors handle:

- JWT attachment
- request logging
- correlation IDs
- global error handling
- automatic token refresh

Business components remain unaware of authentication details.

9.12 Error Handling

Errors are handled consistently across the application.

Business errors are displayed to users using friendly messages.

Unexpected errors are logged and displayed using a generic fallback message.

Examples include:

Validation Errors

Authentication Errors

Authorization Errors

Network Errors

Unexpected Server Errors

The UI should never expose internal backend implementation details.

9.13 Forms

Angular Reactive Forms are used throughout the application.

Reasons include:

- strong typing
- validation
- dynamic controls
- better testability

Typical forms include:

Login

Registration

Dictionary Upload

Profile

Settings

Reactive Forms provide significantly better maintainability than template-driven forms for enterprise applications.

9.14 Component Communication

Communication follows Angular best practices.

Parent → Child

Input Signals

Child → Parent

Output Events

Sibling Components

Shared State Service

Cross-feature communication should be avoided whenever possible.

9.15 UI Design Principles

The frontend follows several design principles.

Single Responsibility

Each component performs one task.

Composition over Inheritance

Complex screens are composed from smaller components.

Container / Presentation Separation

Pages orchestrate.

Components present.

Consistency

The same interaction patterns should appear throughout the application.

Predictability

Users should always understand the current application state.

9.16 Performance Strategy

Performance optimization begins at the architectural level.

The application uses:

- lazy loading
- standalone components
- Angular Signals
- OnPush change detection
- immutable DTOs
- efficient list rendering

Large dictionaries should remain responsive even when containing thousands of cards.

9.17 Accessibility

Accessibility should be considered from the beginning.

The application should support:

- keyboard navigation
- focus management
- screen readers
- semantic HTML
- ARIA attributes
- sufficient color contrast

Accessibility is treated as a quality attribute rather than an optional feature.

9.18 Future Frontend Extensions

The architecture intentionally supports future additions.

Examples include:

Dark Mode

Offline Mode

PWA

Speech Recognition

Text-to-Speech

AI Assistant

Mobile Layout

Tablet Optimization

Multiple Themes

Internationalization

None of these features require changes to the existing feature structure.

9.19 Frontend Architecture Summary

The frontend architecture emphasizes simplicity, modularity and maintainability.

Feature-based organization, Angular Signals, standalone components and lazy loading create a clean separation of responsibilities while avoiding unnecessary complexity.

Pages coordinate user interaction, components remain focused on presentation, services encapsulate communication and the backend remains the single source of business truth.

This architecture provides a scalable foundation capable of supporting future features without sacrificing readability or developer productivity.

10. Authentication & Authorization

10.1 Overview

Authentication and authorization are fundamental architectural concerns rather than implementation details.

The primary objective of the security architecture is to ensure that every request is executed on behalf of an authenticated user while guaranteeing complete isolation between user-owned resources.

Security is implemented centrally using Spring Security and follows modern industry best practices.

The architecture is based on the following principles:

- Stateless authentication
- Ownership-based authorization
- Least privilege
- Secure by default
- Defense in depth

Authentication verifies **who the user is**.

Authorization determines **what the authenticated user is allowed to access**.

These responsibilities remain clearly separated throughout the application.

10.2 Security Architecture

The application consists of three security layers.

Client

|

JWT Authentication

|

Authorization

|

Business Logic

Each request must successfully pass all three layers before reaching the application.

Security failures are rejected as early as possible.

10.3 Authentication Strategy

The application uses JSON Web Tokens (JWT) for stateless authentication.

Two token types are used.

Access Token

Purpose:

Authenticate API requests.

Characteristics:

- Short lifetime
- Signed
- Stateless
- Sent with every API request

Typical lifetime:

15–30 minutes.

Refresh Token

Purpose:

Generate new access tokens without requiring the user to log in again.

Characteristics:

- Long lifetime
- Stored securely
- Revocable
- Rotated after every successful refresh

Typical lifetime:

7–30 days.

10.4 Authentication Flow

The authentication process follows the sequence below.

Login



Username + Password



Spring Security Authentication



JWT Access Token



Refresh Token



Client Stores Tokens



Authenticated Requests

The backend never stores HTTP sessions.

Every request is authenticated independently.

10.5 Login Process

The login workflow consists of the following steps.

1. User submits credentials.
2. Spring Security authenticates the user.
3. Password hash is verified.
4. JWT Access Token is generated.
5. Refresh Token is generated.
6. Refresh Token is persisted.
7. Tokens are returned to the client.

The backend never returns sensitive user information during authentication.

10.6 Password Storage

Passwords are never stored in plaintext.

The application uses BCrypt for password hashing.

The database stores only:

- password hash
- hash algorithm version

Passwords are never:

- logged
- returned by APIs
- cached
- exported

10.7 Password Policy

Passwords should satisfy the following minimum requirements.

Minimum length:

12 characters

Recommended requirements:

- uppercase letter
- lowercase letter
- digit
- special character

Common passwords should be rejected.

Passwords should be validated both on the frontend and backend.

10.8 Refresh Token Rotation

Refresh Tokens are rotated after every successful refresh request.

Process:

Old Refresh Token

↓

Validation

↓

Generate New Pair

↓

Persist New Refresh Token



Invalidate Old Refresh Token

This significantly reduces the impact of token theft.

10.9 Logout Strategy

Logging out invalidates the active Refresh Token.

Access Tokens naturally expire after their configured lifetime.

Logout process:

1. Client sends logout request.
2. Backend invalidates Refresh Token.
3. Client removes locally stored tokens.
4. Authentication session ends.

Logout does not require server-side HTTP sessions.

10.10 Authorization Model

Authorization is ownership-based.

Every persistent resource belongs to exactly one user.

Authorization verifies ownership before any business operation is executed.

Examples:

A user may access:

- their own dictionaries
- their own games
- their own statistics
- their own profile

A user may never access another user's resources.

Ownership checks are mandatory for every business operation.

10.11 Ownership Verification

Ownership is determined through the domain hierarchy.

Authenticated User



Dictionary



Card



Game



Answer



Statistics

The application never trusts identifiers received from the client.

Ownership is always verified using database queries.

Example:

Instead of:

```
findDictionary(id)
```

The application performs:

```
findDictionaryByIdAndUser(id, currentUser)
```

This prevents unauthorized resource access.

10.12 IDOR Protection

One of the primary security concerns is Insecure Direct Object Reference (IDOR).

Example attack:

A malicious user changes:

```
/api/dictionaries/15
```

to

```
/api/dictionaries/16
```

If Dictionary 16 belongs to another user, the backend must return:

HTTP 404

instead of exposing the existence of that resource.

Ownership validation prevents this class of attacks.

10.13 Endpoint Security

Endpoints are grouped by access level.

Public

Examples:

- registration
- login
- refresh token

Authentication is not required.

Authenticated

Examples:

- dictionaries
- games
- statistics
- profile

Authentication is mandatory.

Ownership validation is mandatory.

Administrative

Reserved for future versions.

Examples:

- user management
- system monitoring
- audit reports

Administrative endpoints require dedicated roles.

10.14 Role Model

The initial version intentionally keeps authorization simple.

Roles:

USER

ADMIN (future)

Most authorization decisions rely on ownership rather than role hierarchy.

This significantly reduces complexity.

10.15 File Upload Security

Excel upload introduces additional security considerations.

Every uploaded file must be validated.

Validation includes:

- MIME type
- file extension
- maximum file size
- workbook structure
- worksheet count
- required columns
- malformed content

Executable content is never accepted.

The original uploaded file is not permanently stored.

10.16 Input Validation

Every incoming request undergoes multiple validation stages.

Layer 1

Bean Validation

Examples:

@NotBlank

@NotNull

@Email

Layer 2

Business Validation

Examples:

Dictionary ownership

Duplicate names

Game state

Layer 3

Database Constraints

Examples:

Unique constraints

Foreign keys

Check constraints

Every layer contributes to overall system security.

10.17 HTTP Security Headers

The application should configure modern security headers.

Recommended headers include:

Content-Security-Policy

X-Content-Type-Options

Referrer-Policy

Permissions-Policy

X-Frame-Options

Strict-Transport-Security

These headers significantly reduce browser-based attack vectors.

10.18 CORS Strategy

Only trusted frontend origins should be allowed.

Example:

Development

`http://localhost:4200`

Production

`https://app.deutschlingodeck.com`

Wildcard origins should never be enabled in production.

10.19 CSRF Protection

Since authentication uses JWT rather than server-side sessions, the application is significantly less exposed to classical CSRF attacks.

If Refresh Tokens are stored inside secure HTTP-only cookies, CSRF protection should additionally be enabled for refresh endpoints.

10.20 Rate Limiting

Rate limiting protects against abuse.

Recommended limits include:

Login attempts

Registration

Password reset

Refresh Token endpoint

Dictionary upload

This reduces brute-force attacks and denial-of-service risks.

10.21 Audit Logging

Security-sensitive operations should generate audit events.

Examples include:

Successful login

Failed login

Password change

Dictionary upload

Dictionary deletion

Game completion

Audit logs should contain:

- timestamp
- authenticated user
- operation
- IP address
- correlation ID

Passwords and JWT tokens must never appear in logs.

10.22 Security Configuration

Security configuration should remain centralized.

Responsibilities include:

- authentication filters
- JWT validation
- endpoint authorization
- password encoding
- CORS configuration
- exception handling

Business modules should never contain security configuration code.

10.23 OWASP Considerations

The application is designed in accordance with the OWASP Top 10 recommendations.

Particular attention should be given to:

- Broken Access Control
- Cryptographic Failures
- Injection Attacks
- Insecure Design
- Security Misconfiguration
- Vulnerable Dependencies
- Identification and Authentication Failures

Security should be continuously reviewed throughout the development lifecycle.

10.24 Future Security Enhancements

Future versions may introduce additional security features.

Examples include:

- Multi-Factor Authentication (MFA)
- OAuth2 Login
- OpenID Connect
- Passkeys (WebAuthn)
- Device Management
- Trusted Devices
- Account Activity History
- Login Notifications
- Account Lockout Policies

The current architecture is fully compatible with these future enhancements.

10.25 Security Summary

Security is treated as a first-class architectural concern rather than a collection of isolated implementation details.

Authentication, authorization and ownership verification are clearly separated, enabling a simple yet robust security model.

Stateless JWT authentication, ownership-based authorization, centralized validation and comprehensive audit logging provide a secure foundation suitable for production deployments while remaining flexible enough to support future enterprise-grade security requirements.

11. Dictionary Import Pipeline

11.1 Overview

Dictionary import is one of the core business processes of DeutschLingoDeck. Although it may appear to be a simple file upload, it is intentionally designed as a structured processing pipeline consisting of multiple independent stages.

Each stage has a single responsibility and validates the input before passing it to the next stage.

This approach improves maintainability, simplifies testing and provides detailed error reporting while ensuring that invalid data never reaches the persistence layer.

The pipeline follows the **Fail Fast** principle. If any stage detects an unrecoverable error, processing stops immediately and the user receives a comprehensive validation report.

11.2 Pipeline Overview

The complete import process consists of the following stages.

Excel Upload

|



Technical Validation

|



File Parsing

|



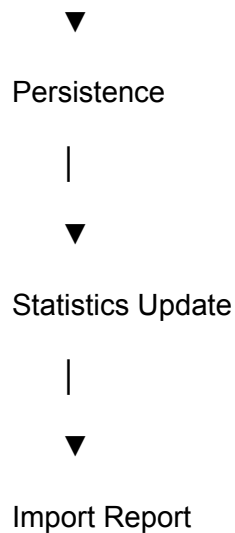
Business Validation

|



Domain Object Creation

|



Each stage produces a clearly defined output that becomes the input of the following stage.

11.3 Pipeline Goals

The import pipeline is designed to achieve the following objectives:

- ensure data quality
- provide detailed validation feedback
- guarantee transactional consistency
- prevent duplicate data
- isolate processing responsibilities
- simplify future extension

The pipeline should remain independent of the REST layer so that future import sources (REST, CLI, scheduled jobs or external APIs) can reuse the same processing logic.

11.4 Stage 1 – File Upload

The import process begins when the user uploads an Excel workbook.

Responsibilities include:

- receiving multipart file
- checking request size
- authenticating user
- verifying ownership
- creating import request

No parsing occurs at this stage.

The uploaded file is treated as an opaque binary object.

11.5 Stage 2 – Technical Validation

Technical validation verifies whether the uploaded file can be processed.

Typical checks include:

- supported file extension
- valid MIME type
- maximum file size
- workbook integrity
- readable workbook
- expected worksheet count

If any validation fails, processing stops immediately.

Example validation errors:

- unsupported file type
- corrupted workbook
- encrypted workbook
- empty workbook
- file exceeds maximum size

Technical validation intentionally ignores business rules.

11.6 Stage 3 – Excel Parsing

Once the workbook passes technical validation, Apache POI extracts its contents.

Responsibilities include:

- locating worksheets
- reading rows
- reading columns
- preserving row numbers
- converting Excel cells into raw import objects

No persistence occurs during parsing.

The parser performs only data extraction.

11.7 Import Data Model

After parsing, every row is represented by an intermediate import object.

Example:

ImportedCardRow

rowNumber

sourceText

translation

article

cardType

example

notes

These objects remain independent from JPA entities.

This separation significantly improves validation and testing.

11.8 Stage 4 – Business Validation

Business validation verifies whether imported data satisfies application rules.

Examples include:

Required values

Duplicate cards

Duplicate translations

Unsupported card types

Invalid articles

Empty source text

Empty translation

Maximum text length

Unknown language codes

Unlike technical validation, business validation understands the application domain.

Validation errors are accumulated so the user receives a complete report instead of fixing one error at a time.

11.9 Validation Report

Every validation error includes:

- worksheet
- row number
- column
- validation rule
- human-readable description

Example:

Row: 27

Column: Translation

Error:

Translation must not be empty.

This allows users to correct import errors efficiently.

11.10 Stage 5 – Domain Object Creation

After successful validation, the pipeline transforms import objects into domain entities.

Examples:

ImportedCardRow

↓

Card Entity

ImportedDictionary

↓

Dictionary Entity

The mapping process should not perform validation because all validation has already completed.

11.11 Stage 6 – Persistence

Persistence occurs inside a single database transaction.

Typical workflow:

Create Dictionary

↓

Persist Cards

↓

Update Card Count

↓

Create Import Log

↓

Commit Transaction

If any persistence step fails, the entire transaction is rolled back.

Partial dictionary imports are never allowed.

11.12 Stage 7 – Statistics Update

After successful persistence, initial statistics are generated.

Examples include:

Dictionary card count

Import timestamp

Import duration

Import version

The system intentionally avoids generating learning statistics during import because no learning activity has yet occurred.

11.13 Stage 8 – Import Report

The final pipeline stage generates a summary for the user.

Typical information includes:

Dictionary name

Imported cards

Ignored rows

Warnings

Processing duration

Import timestamp

Import version

Users receive immediate feedback confirming whether the import completed successfully.

11.14 Import Versioning

The Excel template is versioned.

Each uploaded workbook contains an internal template version.

Example:

LINGODECK_TEMPLATE_V1

The backend selects the appropriate parser based on this version.

Future versions of the template can therefore coexist without breaking backward compatibility.

11.15 Idempotency

The import process should be idempotent whenever possible.

Repeated imports of the same dictionary should not unintentionally create duplicate data.

Possible strategies include:

- duplicate detection
- checksum comparison
- dictionary versioning
- overwrite confirmation

The exact strategy may evolve in future releases.

11.16 Error Handling Strategy

Import failures are classified into three categories.

Technical Errors

Examples:

Unreadable workbook

Corrupted file

Network interruption

Unexpected parser failure

Validation Errors

Examples:

Missing translation

Duplicate card

Invalid article

Missing worksheet

System Errors

Examples:

Database unavailable

Transaction failure

Unexpected runtime exception

Each category produces a different response suitable for both users and developers.

11.17 Performance Considerations

The import pipeline should efficiently process large dictionaries.

Recommended optimizations include:

- streaming workbook reading
- batch inserts
- prepared statements
- minimizing object creation
- avoiding unnecessary database queries

The initial implementation should comfortably support dictionaries containing several thousand cards.

11.18 Future Extensions

The pipeline is intentionally extensible.

Future import sources may include:

CSV

JSON

XML

REST APIs

Google Sheets

AI-generated dictionaries

External vocabulary providers

Only the parsing stage should change.

Business validation and persistence remain unchanged.

11.19 Architectural Benefits

The pipeline architecture provides several important advantages.

Responsibilities remain clearly separated.

Validation logic becomes reusable.

Business rules remain independent from Excel parsing.

Future import formats can reuse the same validation pipeline.

Testing becomes significantly easier because every processing stage can be tested independently.

Most importantly, the architecture follows the Single Responsibility Principle while remaining simple enough to understand and maintain.

11.20 Dictionary Import Summary

The Dictionary Import Pipeline transforms a seemingly simple file upload into a structured business workflow.

Each processing stage performs one clearly defined responsibility and passes validated data to the next stage.

This design minimizes coupling, improves error reporting, guarantees transactional consistency and provides a robust foundation for future import formats and advanced processing capabilities.

Rather than treating import as a controller operation, the system models it as a first-class business process, making it one of the most maintainable and extensible parts of the entire application.

12. Game Engine

12.1 Overview

The Game Engine is the central business component of DeutschLingoDeck.

Its responsibility is to orchestrate the complete learning session, from the moment a user starts a game until the session is completed and all statistics have been updated.

Unlike the frontend, which is responsible only for presenting flashcards and collecting user input, the Game Engine owns all business rules related to gameplay.

The Game Engine is intentionally designed as a stateful business workflow implemented on top of a stateless REST architecture. Although HTTP requests are independent, the backend maintains the current game state inside the database.

This design guarantees that users can safely refresh the browser, reconnect after temporary network interruptions or continue a game from another device without losing progress.

12.2 Responsibilities

The Game Engine is responsible for:

- creating new games
- selecting cards
- tracking game progress
- accepting answers
- coordinating answer validation
- updating statistics
- calculating scores
- finishing games
- handling abandoned games

The Game Engine intentionally delegates specialized business logic to dedicated Domain Services.

12.3 Game Lifecycle

Every learning session follows the same lifecycle.

Create Game

|



Initialize Session



Select Next Card



Display Card



Receive Answer



Validate Answer



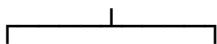
Store Answer



Update Statistics



Next Card?



Yes

No



Next Finish Game

Card

Each transition represents a well-defined business operation.

12.4 Game Creation

A new game begins when the user selects one of their dictionaries.

The backend creates a new Game entity containing:

- owner
- selected dictionary
- creation timestamp
- status
- total number of cards
- current progress

At this point no answers have yet been submitted.

The game immediately becomes the active learning session.

12.5 Game State

The Game entity represents the current state of the session.

Possible states include:

CREATED

STARTED

IN_PROGRESS

FINISHED

ABANDONED

State transitions are strictly controlled.

Illegal transitions are rejected.

Examples:

FINISHED → STARTED

ABANDONED → IN_PROGRESS

FINISHED → CREATED

These transitions are not permitted.

12.6 Card Selection

The Card Selection Service determines which card should be presented next.

The first production version intentionally uses a deterministic strategy.

Example:

Cards appear in the order defined by the imported dictionary.

Future versions may replace this algorithm without modifying the Game Engine.

Examples include:

- random selection
- weighted randomness
- spaced repetition
- adaptive learning
- AI-generated review order

The Game Engine never assumes how cards are selected.

It simply requests the next card from the Card Selection Service.

12.7 Current Card

At any point during the session exactly one card is active.

The backend returns only the information required for display.

Example response:

```
{  
  "cardId": 152,  
  "sourceText": "table",  
  "cardType": "NOUN",  
  "example": "The table is made of wood."  
}
```

Correct answers are never sent to the frontend before validation.

This prevents accidental disclosure through browser developer tools.

12.8 Answer Submission

When the user submits an answer, the backend performs the following workflow.

Receive Request

|



Verify Ownership

|



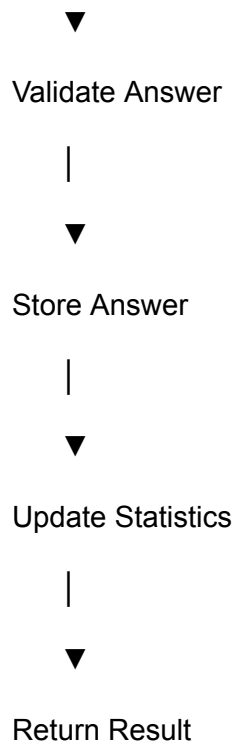
Load Active Game

|



Load Active Card

|



Each step executes inside a single database transaction.

12.9 Game Session Integrity

The backend validates every submitted answer.

Validation includes:

- game ownership
- game status
- active card
- duplicate submission
- answer format

An answer cannot be submitted if:

- the game has finished
- the game was abandoned
- the card does not belong to the game
- the game belongs to another user

These checks prevent inconsistent learning history.

12.10 Progress Tracking

Progress is maintained throughout the session.

Examples include:

Current card

Answered cards

Remaining cards

Elapsed time

Correct answers

Incorrect answers

Completion percentage

Progress is recalculated after every submitted answer.

12.11 Score Calculation

The first production version uses a straightforward scoring model.

Score =

Correct Answers

÷

Answered Cards

×

100

This intentionally separates scoring from learning statistics.

Future versions may introduce weighted scoring based on:

- response time
- difficulty
- streak
- confidence
- spaced repetition

12.12 Timer

Each answer records the response time.

The frontend measures the elapsed time between displaying the card and submitting the answer.

The measured duration is sent to the backend together with the answer.

The backend validates that the value is reasonable before persisting it.

Response time may later influence learning algorithms.

12.13 Session Recovery

Games remain persistent.

If the browser is refreshed:

- the active game remains stored
- answered cards remain stored
- progress remains stored

The frontend simply requests the current game state and continues.

No learning progress is lost.

12.14 Game Completion

A game completes when:

- all cards have been answered

or

- the user explicitly finishes the session.

Completion performs the following actions.

Calculate Duration

|



Finalize Statistics

|



Store Finish Time

|



Mark Game Finished

|



Generate Summary

Finished games become immutable.

12.15 Abandoned Games

A user may abandon a session before completion.

The Game Engine records:

- finish timestamp
- completion percentage
- answered cards
- elapsed time

Historical answers remain preserved.

Abandoned sessions still contribute to learning history.

12.16 Game Summary

After completion the backend generates a summary.

Typical information includes:

Total cards

Correct answers

Incorrect answers

Accuracy

Duration

Average response time

Most difficult cards

Best performance

Future versions may include AI-generated recommendations.

12.17 Concurrency

Only one active game per user is permitted.

If a user attempts to start another game while one is already active, the backend should either:

- resume the existing session

or

- require explicit confirmation before abandoning the current game.

This prevents inconsistent statistics.

12.18 Transaction Boundaries

The following operations execute atomically.

Game Creation

Answer Submission

Game Completion

Game Abandonment

Each transaction guarantees database consistency.

No partially completed business operation is ever persisted.

12.19 Future Extensions

The Game Engine has been intentionally designed to support future enhancements.

Potential additions include:

- spaced repetition scheduling
- adaptive difficulty
- AI-generated hints
- pronunciation evaluation
- speech recognition
- multiplayer learning
- competitive game modes
- achievements
- daily challenges
- review sessions
- timed quizzes

The architecture already isolates card selection and answer validation into dedicated services, allowing these features to be introduced without modifying the overall workflow.

12.20 Architectural Benefits

Separating the Game Engine from individual domain services provides several important advantages.

The Game Engine focuses exclusively on orchestrating the learning session.

Card selection, answer validation and statistics aggregation remain independent business capabilities.

This separation improves maintainability, simplifies testing and allows individual algorithms to evolve independently.

Future learning algorithms can therefore be introduced without rewriting the core game workflow.

12.21 Game Engine Summary

The Game Engine represents the central orchestration layer of the learning process.

It coordinates every phase of a learning session while delegating specialized business rules to dedicated domain services.

This design creates a clear separation between orchestration and business logic, ensuring transactional consistency, robust session recovery and long-term extensibility.

Rather than being a monolithic service containing all gameplay logic, the Game Engine acts as a coordinator that composes specialized services into a complete learning experience.

13. Answer Validation Engine

13.1 Overview

The Answer Validation Engine is responsible for determining whether a user's response is correct.

Unlike simple string comparison, answer validation is treated as an independent business domain. The validation process is expected to evolve continuously as the application introduces new learning modes, AI-assisted evaluation and support for additional languages.

For this reason, answer validation is intentionally isolated from the Game Engine.

The Game Engine orchestrates the learning session, while the Answer Validation Engine decides whether a submitted answer satisfies the expected learning objective.

This separation follows the **Single Responsibility Principle** and significantly improves maintainability.

13.2 Architectural Responsibilities

The Answer Validation Engine is responsible for:

- input normalization
- answer comparison
- article validation
- alternative answer matching
- sentence comparison
- typo detection
- validation result generation

It is **not** responsible for:

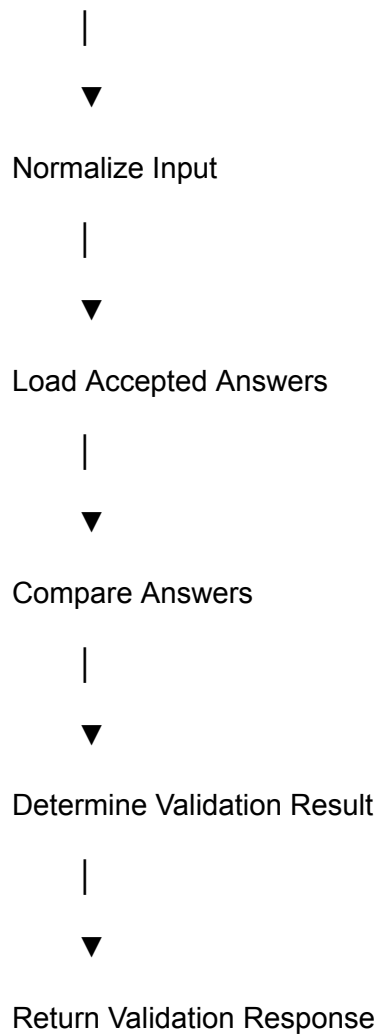
- storing answers
- updating statistics
- progressing the game
- communicating with REST controllers

Those responsibilities belong to other application services.

13.3 Validation Pipeline

Every submitted answer passes through a structured validation pipeline.

Receive User Answer

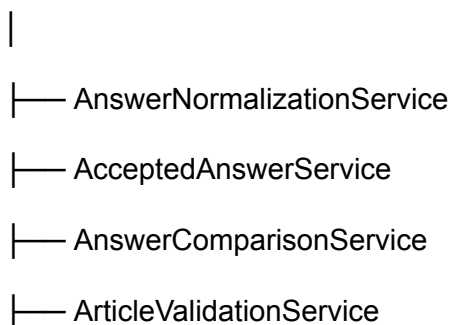


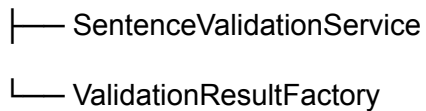
Each stage performs one well-defined task.

13.4 Validation Components

The validation engine consists of several independent services.

AnswerValidationService





Each component encapsulates one business capability.

This modular structure makes it easy to extend validation rules without affecting existing functionality.

13.5 Input Normalization

Before comparing answers, both the user's input and the expected answer are normalized.

Normalization removes irrelevant differences that should not influence correctness.

Typical normalization steps include:

- trimming leading and trailing whitespace
- collapsing multiple spaces
- converting text to lowercase
- normalizing umlauts
- normalizing German sharp S (ß)
- removing redundant punctuation
- normalizing quotation marks
- removing invisible Unicode characters

Example:

Input:

Der Tisch

Normalized:

der tisch

Normalization ensures that cosmetic formatting differences do not affect validation.

13.6 Accepted Answers

Each card may define multiple accepted answers.

Example:

Source:

car

Accepted answers:

Auto

Wagen

PKW

The validation engine compares the normalized user input against all accepted answers.

This approach avoids unnecessary duplication of cards while supporting legitimate language variations.

13.7 Card-Type-Specific Validation

Different card types require different validation strategies.

Noun

Validation verifies:

- correct article
- correct noun
- spelling

Example:

Expected:

der Tisch

User:

die Tisch

Result:

WRONG_ARTICLE

Word

Simple normalized comparison.

Phrase

Comparison ignores insignificant whitespace and punctuation.

Sentence

Sentence validation is more sophisticated.

Possible checks include:

- exact match
- normalized match
- punctuation tolerance
- optional synonyms
- future AI evaluation

Different card types therefore remain independent from one another.

13.8 Validation Result

Validation does not return only a Boolean value.

Instead, it returns a structured business result.

Possible values include:

CORRECT

WRONG_ARTICLE

WRONG_TRANSLATION

TYPO

MISSING_WORD

EXTRA_WORD

PARTIALLY_CORRECT

WRONG_SENTENCE

Returning detailed validation results enables significantly richer user feedback.

13.9 Validation Response

The validation response contains more than the validation result.

Example:

```
{  
  "result": "WRONG_ARTICLE",  
  "correct": false,  
  "expectedAnswer": "der Tisch",  
  "givenAnswer": "die Tisch",
```

```
"message": "Incorrect article."
}
```

The frontend uses this response to present meaningful feedback to the learner.

13.10 Typo Detection

Future versions may tolerate minor spelling mistakes.

Instead of requiring perfect string equality, the engine may calculate edit distance.

Example:

Expected:

Krankenhaus

User:

Krankenhuas

Rather than returning **WRONG_TRANSLATION**, the engine may classify the answer as:

TYPO

The exact tolerance threshold should remain configurable.

13.11 Sentence Comparison

Sentence validation differs significantly from single-word validation.

Future comparison strategies may include:

- normalized comparison
- token comparison
- word order analysis
- optional punctuation

- synonym detection
- grammar evaluation

The architecture intentionally isolates sentence validation to avoid increasing the complexity of simpler validation strategies.

13.12 AI-Assisted Validation

The architecture intentionally prepares the validation engine for AI integration.

Future versions may delegate sentence evaluation to a Large Language Model.

Possible workflow:

Normalize

↓

Rule-Based Validation

↓

Confident?

↓

Yes → Return Result

↓

No



AI Evaluation



Merge Results

Rule-based validation remains the default.

AI becomes an optional secondary validation strategy rather than replacing deterministic business rules.

13.13 Validation Configuration

Validation rules should remain configurable.

Examples include:

- case sensitivity
- punctuation tolerance
- typo tolerance
- article enforcement
- whitespace normalization
- synonym support

Configuration should be managed centrally rather than hardcoded inside services.

13.14 Performance Considerations

Validation is executed for every submitted answer.

Therefore:

- unnecessary database queries should be avoided
- accepted answers should be retrieved efficiently
- normalization should minimize object creation
- expensive AI evaluation should execute only when necessary

The validation engine should remain deterministic and fast for the overwhelming majority of requests.

13.15 Testing Strategy

Every validation rule must be independently testable.

Unit tests should cover:

- normalization
- article validation
- accepted answers
- sentence comparison
- validation results
- edge cases

Business validation should never rely solely on integration testing.

13.16 Future Extensions

The current architecture intentionally supports future capabilities.

Examples include:

- grammar evaluation
- pronunciation scoring
- speech recognition
- contextual hints
- AI-generated explanations
- confidence scoring
- adaptive validation rules
- language-specific validation modules

These additions can be introduced without modifying the public API of the validation engine.

13.17 Architectural Benefits

Treating answer validation as an independent business domain provides several advantages.

Validation logic remains isolated from gameplay.

New validation algorithms can be introduced without affecting the Game Engine.

Language-specific rules remain modular.

AI integration becomes optional rather than mandatory.

Testing becomes significantly simpler because every validation rule can be verified independently.

13.18 Answer Validation Engine Summary

The Answer Validation Engine is designed as an independent domain responsible for evaluating user responses.

By separating normalization, comparison and rule evaluation into dedicated services, the architecture remains modular, extensible and easy to maintain.

The engine supports deterministic validation today while providing a natural extension point for future AI-assisted evaluation, advanced grammar checking and adaptive learning algorithms.

Rather than comparing two strings, the system evaluates answers through a configurable business pipeline capable of evolving alongside the application without disrupting existing functionality.

14. Statistics Engine

14.1 Overview

The Statistics Engine is responsible for collecting, maintaining and exposing learning statistics throughout the application.

Unlike the Game Engine, which orchestrates a single learning session, the Statistics Engine provides a long-term view of the user's learning progress.

The engine continuously aggregates historical learning events into efficient read models while preserving the complete event history.

The design follows a fundamental architectural principle:

Historical events are immutable. Statistics are derived.

This principle guarantees that aggregated values can always be recalculated if new algorithms or business rules are introduced.

14.2 Responsibilities

The Statistics Engine is responsible for:

- updating learning statistics
- aggregating historical answers
- generating dashboard metrics
- calculating success rates
- identifying difficult cards
- identifying mastered cards
- exposing statistics through REST APIs

The Statistics Engine is **not** responsible for:

- validating answers
- orchestrating games
- selecting cards
- importing dictionaries

These responsibilities belong to other business modules.

14.3 Event-Driven Statistics

Every submitted answer represents a business event.

Instead of directly calculating all dashboard values on demand, the system stores every answer permanently and updates aggregated statistics immediately.

The workflow is as follows.

Answer Submitted

|



Persist Historical Answer

|



Update Aggregated Statistics

|



Dashboard Ready

This architecture combines complete historical traceability with excellent read performance.

14.4 Levels of Aggregation

Statistics are maintained on three different levels.

Card Statistics

Represents learning progress for one specific card.

Typical metrics include:

- total appearances
- correct answers
- incorrect answers
- success rate
- average response time
- last appearance
- last correct answer

Dictionary Statistics

Represents learning progress across an entire dictionary.

Typical metrics include:

- total cards
- completed games
- average accuracy
- mastered cards
- difficult cards
- total study time

User Statistics

Represents overall learning progress.

Examples include:

- total games
- total answers
- total study time
- overall accuracy
- learning streak
- average daily activity

These statistics summarize the user's complete learning history.

14.5 Update Workflow

Statistics are updated immediately after every successfully validated answer.

The workflow is executed inside the same database transaction.

Store Answer

|



Update Card Statistics

|



Update Dictionary Statistics

|



Update User Statistics

|



Commit Transaction

This guarantees that historical events and aggregated statistics always remain consistent.

14.6 Card Statistics

Card statistics provide the most detailed level of learning information.

Typical attributes include:

- appearances
- correct answers
- incorrect answers
- accuracy
- average response time
- fastest response
- slowest response
- last reviewed
- last correct answer

Card statistics are always associated with:

- one user
- one dictionary
- one card

This prevents accidental mixing of statistics across users or dictionaries.

14.7 Dictionary Statistics

Dictionary statistics summarize the user's progress within a single dictionary.

Typical metrics include:

- games played
- cards reviewed
- average accuracy
- completion percentage
- average session duration
- average response time

Dictionary statistics provide a high-level overview while remaining independent from individual game sessions.

14.8 User Statistics

User statistics summarize all learning activity.

Examples include:

Total Games

Total Answers

Total Correct Answers

Total Incorrect Answers

Overall Accuracy

Total Learning Time

Current Learning Streak

Longest Learning Streak

Average Daily Study Time

These values support dashboards and long-term progress visualization.

14.9 Learning Streak

The application tracks continuous learning activity.

Example:

Monday

Tuesday

Wednesday

Thursday

Learning streaks motivate consistent practice and may later be used for achievements or gamification.

A streak increases when the user completes at least one learning session during a calendar day.

14.10 Difficult Cards

The engine identifies cards requiring additional practice.

Examples include:

Low accuracy

High response time

Frequent mistakes

Repeated incorrect articles

These cards may later receive higher priority in adaptive learning algorithms.

14.11 Mastered Cards

Cards may be considered mastered once predefined criteria have been satisfied.

Possible criteria include:

- minimum number of appearances
- minimum success rate
- consecutive correct answers
- recent successful review

The exact algorithm remains configurable.

Mastered cards remain available but may appear less frequently in future learning sessions.

14.12 Dashboard Metrics

The Statistics Engine provides optimized data for dashboards.

Examples include:

Overall Progress

Learning Today

Weekly Progress

Most Difficult Cards

Most Successful Dictionary

Average Accuracy

Recent Activity

The dashboard never performs expensive calculations on demand.

Instead, it consumes pre-aggregated data.

14.13 Historical Analytics

Historical answers enable advanced analysis beyond simple counters.

Examples include:

Accuracy over time

Response time trends

Learning frequency

Improvement curves

Dictionary popularity

Daily activity

Because every answer is preserved, historical reports can always be regenerated.

14.14 Future Spaced Repetition Support

The current architecture intentionally prepares the Statistics Engine for spaced repetition algorithms.

Future metrics may include:

Ease Factor

Review Interval

Next Review Date

Retention Score

Memory Strength

The existing event history already contains all information required to calculate these values.

14.15 AI Analytics

Future versions may enrich statistics using AI.

Examples include:

Most common grammar mistakes

Most common article mistakes

Weak vocabulary categories

Personalized recommendations

Learning style analysis

Estimated vocabulary level

AI-generated study plans

The current statistics model provides an excellent foundation for these future capabilities.

14.16 Performance Strategy

The Statistics Engine prioritizes fast read performance.

Expensive calculations should occur only when historical events are written.

Dashboard queries should rely primarily on aggregated statistics.

This architecture minimizes CPU usage while keeping user interactions highly responsive.

14.17 Testing Strategy

Statistics calculations must be deterministic.

Tests should verify:

- counter updates
- percentages
- response time calculations
- streak calculation
- mastered card detection
- difficult card detection

Historical events should always produce identical aggregated statistics regardless of processing order.

14.18 Future Extensions

Future versions may introduce:

Achievements

Badges

Leaderboards

Weekly Challenges

Monthly Reports

Email Progress Reports

Personal Learning Goals

Calendar Heat Maps

Knowledge Graphs

These features can all be implemented using the existing event history.

14.19 Architectural Benefits

Separating historical events from aggregated statistics provides several important advantages.

Historical data remains immutable.

Statistics remain fast.

Algorithms remain replaceable.

Analytics become significantly richer.

The application gains long-term flexibility without sacrificing performance.

This architecture follows the Command Query Responsibility Segregation (CQRS) principle at a conceptual level by separating write models (historical answers) from read models (aggregated statistics), while remaining within a single relational database.

14.20 Statistics Engine Summary

The Statistics Engine transforms raw learning events into meaningful insights.

Rather than treating statistics as simple counters, the architecture models them as optimized read models derived from immutable historical events.

This design ensures consistency, scalability and extensibility while providing an excellent foundation for future adaptive learning algorithms, AI-assisted recommendations and advanced analytical features.

The Statistics Engine therefore becomes not only a reporting component, but one of the central business capabilities of the entire application.

15. REST API Design

15.1 Overview

The REST API represents the communication contract between the Angular frontend and the Spring Boot backend.

It is the only mechanism through which clients interact with the business domain. The frontend never communicates directly with the database and never bypasses the backend.

The API follows established REST principles while remaining pragmatic. The primary design goals are:

- simplicity
- consistency
- predictability
- discoverability
- backward compatibility
- strong typing

The API is versioned from the beginning to allow future evolution without breaking existing clients.

Base URL:

/api/v1

15.2 API Design Principles

The API follows several design principles.

Resource-Oriented Design

Endpoints expose business resources rather than technical operations.

Examples:

/users

/dictionaries

/cards

/games

/statistics

instead of

/createDictionary

/getStatistics

/updateGame

HTTP methods express the requested operation.

Consistent Naming

Resource names are plural.

Examples:

/api/v1/dictionaries

/api/v1/games

/api/v1/statistics

Resource identifiers appear only once.

Good:

GET /api/v1/dictionaries/{dictionaryId}

Avoid:

GET /api/v1/getDictionaryById/{dictionaryId}

Stateless Communication

Every request contains all information required for processing.

The backend never relies on server-side HTTP sessions.

Authentication is provided through JWT Access Tokens.

15.3 API Versioning

Versioning is included in the URI.

Example:

/api/v1

Future versions may introduce:

/api/v2

The initial version should avoid breaking API contracts.

Backward compatibility is preferred whenever possible.

15.4 Authentication Endpoints

Authentication resources.

Method	Endpoint	Description
--------	----------	-------------

POST	/auth/register	Register new user
POST	/auth/login	Authenticate user
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Invalidate refresh token
GET	/auth/me	Retrieve current user

15.5 Dictionary Endpoints

Dictionary management.

Method	Endpoint	Description
GET	/dictionaries	List user dictionaries
POST	/dictionaries/import	Upload Excel dictionary
GET	/dictionaries/{id}	Retrieve dictionary
PUT	/dictionaries/{id}	Update dictionary metadata
DELETE	/dictionaries/{id}	Soft delete dictionary

Cards remain subordinate resources.

Examples:

GET /dictionaries/{id}/cards

GET /dictionaries/{id}/cards/{cardId}

15.6 Game Endpoints

Game lifecycle.

Method	Endpoint	Description
POST	/games	Start new game
GET	/games/{id}	Retrieve game
POST	/games/{id}/answers	Submit answer
POST	/games/{id}/finish	Finish game
POST	/games/{id}/abandon	Abandon game
GET	/games/{id}/summary	Retrieve summary

Only active games may receive answers.

Finished games become read-only.

15.7 Statistics Endpoints

Statistics retrieval.

Method	Endpoint	Description
GET	/statistics	User dashboard
GET	/statistics/cards	Card statistics
GET	/statistics/dictionaries	Dictionary statistics
GET	/statistics/history	Learning history

Statistics endpoints are read-only.

15.8 Profile Endpoints

User profile management.

Method	Endpoint	Description
GET	/profile	Retrieve profile
PUT	/profile	Update profile
PUT	/profile/password	Change password

15.9 HTTP Status Codes

The API consistently uses standard HTTP status codes.

Successful responses:

200 OK

201 Created

204 No Content

Client errors:

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

422 Unprocessable Entity

Server errors:

500 Internal Server Error

503 Service Unavailable

The same business situation should always return the same status code.

15.10 Request DTOs

Incoming requests are represented by immutable DTOs.

Example:

```
{  
  "dictionaryId": 15,  
  "gameMode": "STANDARD"  
}
```

Request DTOs never expose persistence details.

15.11 Response DTOs

Responses contain only information required by the client.

Example:

```
{  
  "gameId": 42,  
  "status": "IN_PROGRESS",  
  "currentCard": {  
    "id": 81,  
    "sourceText": "house",  
    "cardType": "NOUN"  
  }  
}
```

Internal entity identifiers remain implementation details whenever possible.

15.12 Pagination

Collection endpoints should support pagination.

Example:

GET /games?page=0&size=20

Future enhancements may include:

- sorting
- filtering
- searching

The API should remain efficient regardless of data volume.

15.13 Filtering

Filtering follows standard query parameters.

Examples:

GET /games?status=FINISHED

GET /games?dictionaryId=5

GET /statistics?from=2026-01-01&to=2026-12-31

Filtering should remain intuitive and composable.

15.14 Validation Errors

Validation errors follow a standardized format.

Example:

{

```
"timestamp": "2026-07-02T14:32:11Z",  
"status": 400,  
"code": "VALIDATION_ERROR",  
"message": "Validation failed.",  
"errors": [  
  {  
    "field": "email",  
    "message": "Email address is invalid."  
  }  
]  
}
```

Clients can therefore display field-level validation messages consistently.

15.15 Business Errors

Business errors are represented separately from validation errors.

Example:

```
{  
  "timestamp": "2026-07-02T14:35:10Z",  
  "status": 409,  
  "code": "DICTIONARY_ALREADY_EXISTS",  
  "message": "A dictionary with this name already exists."  
}
```

Business error codes remain stable across API versions.

15.16 Correlation IDs

Every request receives a Correlation ID.

Example response header:

X-Correlation-Id: 8b4c44e2-12aa-4f59-a96f-f73b7d3dbd18

This identifier simplifies troubleshooting across logs.

15.17 Idempotency

Certain operations should be idempotent.

Examples:

PUT

DELETE

Password updates

Profile updates

Repeated execution should produce the same final state.

Game answer submission intentionally remains non-idempotent because each answer represents a historical event.

15.18 OpenAPI Documentation

The API should be documented using OpenAPI.

Swagger UI should be available in development environments.

Documentation should include:

- endpoint descriptions
- request schemas
- response schemas
- authentication requirements
- example payloads
- error responses

The OpenAPI specification becomes the authoritative API contract.

15.19 API Evolution Strategy

Breaking changes should be avoided.

Preferred evolution strategies include:

- adding optional fields
- introducing new endpoints
- deprecating obsolete functionality
- creating a new API version only when necessary

API stability is considered a long-term architectural objective.

15.20 REST API Summary

The REST API provides a clean, predictable and versioned communication layer between the frontend and backend.

Resource-oriented endpoints, immutable DTOs, standardized error responses and consistent HTTP semantics create a robust API contract that is easy to consume, test and maintain.

By treating the API as a first-class architectural component rather than a transport mechanism, the application remains well prepared for future clients such as mobile applications, desktop applications or external integrations.

16. Docker & Deployment Architecture

16.1 Overview

DeutschLingoDeck is designed to be deployed as a containerized application from the very beginning.

Although the initial deployment targets a single Docker Compose environment, the architecture intentionally follows practices that closely resemble production deployments. This approach minimizes the gap between development and production environments while simplifying future migration to container orchestration platforms such as Kubernetes.

Every application component executes inside its own isolated container.

The initial deployment consists of four services:

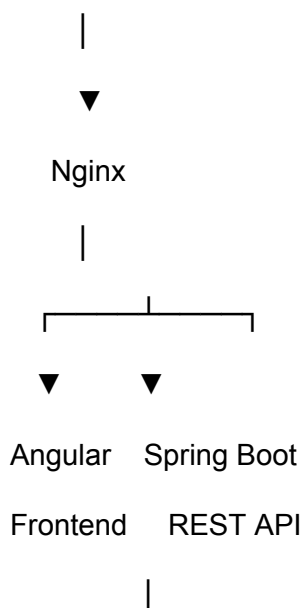
- Angular Frontend
- Spring Boot Backend
- PostgreSQL Database
- Nginx Reverse Proxy

Each service has a clearly defined responsibility and communicates only through well-defined network interfaces.

16.2 Deployment Overview

The initial deployment architecture is shown below.

Internet / Browser





PostgreSQL

The browser communicates exclusively with Nginx.

Nginx serves the Angular application and proxies all API requests to the backend.

The database is never exposed directly to clients.

16.3 Container Responsibilities

Angular Container

Responsibilities include:

- serving the Single Page Application
- loading static assets
- client-side routing
- interacting with REST APIs

The Angular container never communicates directly with PostgreSQL.

Nginx Container

Responsibilities include:

- serving static frontend assets
- reverse proxy
- HTTPS termination
- compression
- caching
- security headers
- request forwarding

Nginx acts as the public entry point of the system.

Spring Boot Container

Responsibilities include:

- REST API
- authentication
- authorization
- business logic
- validation
- persistence
- statistics
- dictionary import

The backend communicates exclusively with PostgreSQL.

PostgreSQL Container

Responsibilities include:

- persistent storage
- ACID transactions
- indexing
- constraints
- backups

The database accepts connections only from trusted internal services.

16.4 Docker Network

All services communicate over an isolated Docker bridge network.

frontend-network

Angular

Nginx

Spring Boot

PostgreSQL

Internal services communicate using Docker service names instead of IP addresses.

Example:

spring-backend

postgres

No internal service should depend on fixed IP addresses.

16.5 Docker Volumes

Persistent data is stored using Docker volumes.

Volumes include:

PostgreSQL Data

Application Logs

Uploaded Files (optional)

Volumes guarantee that application data survives container restarts.

Containers remain stateless whenever possible.

16.6 Environment Configuration

Environment-specific configuration is externalized.

Typical configuration includes:

Database URL

Database Credentials

JWT Secret

JWT Expiration

Refresh Token Expiration

File Upload Limits

Logging Level

Allowed Origins

Configuration is injected through environment variables.

Business logic never depends on hardcoded values.

16.7 Docker Compose

Docker Compose orchestrates the complete application stack.

Each service defines:

- image
- build configuration
- ports
- networks
- environment variables
- health checks
- dependencies
- restart policy

Starting the complete application should require only a single command.

Example:

```
docker compose up
```

Stopping the application should also stop all dependent services gracefully.

16.8 Startup Sequence

Application startup follows a predictable order.

PostgreSQL

↓

Flyway Migration

↓

Spring Boot

↓

Angular Build

↓

Nginx Ready

The backend should never attempt to start before the database becomes healthy.

Likewise, Nginx should not expose the frontend until backend services are available.

16.9 Health Checks

Every service exposes a health endpoint.

Examples include:

Spring Boot

/actuator/health

PostgreSQL

Database connection health

Nginx

HTTP availability

Docker Compose should use health checks rather than simple startup ordering.

16.10 Configuration Profiles

The backend supports multiple runtime profiles.

Examples include:

Development

Testing

Production

Each profile defines:

- logging
- database
- security
- debugging
- monitoring

The application behaves consistently while allowing environment-specific optimization.

16.11 Reverse Proxy

Nginx performs request routing.

Typical routing rules include:

/

↓

Angular SPA

/api/**

↓

Spring Boot

/assets/**



Static Files

This keeps the frontend unaware of backend deployment details.

16.12 HTTPS Strategy

Production deployments should always use HTTPS.

TLS termination is handled by Nginx.

Future deployments may integrate:

- Let's Encrypt
- Cloudflare
- Reverse proxy load balancers

The backend itself remains protocol-agnostic.

16.13 Logging

Application logs remain separated.

Examples:

Angular Logs

Spring Boot Logs

Nginx Access Logs

Nginx Error Logs

PostgreSQL Logs

Structured logging simplifies troubleshooting and monitoring.

16.14 Backup Strategy

Persistent data should be backed up regularly.

Critical backup targets include:

PostgreSQL Database

Dictionary Metadata

Import Logs

Configuration

Uploaded dictionary files (optional)

Application binaries should never require backup because they are reproducible from source code.

16.15 Deployment Strategy

Deployments should be repeatable.

Every deployment consists of:

Build

↓

Container Image

↓

Docker Registry

↓

Deployment

↓

Health Verification

↓

Traffic Routing

Rollback should always remain possible.

16.16 Future Kubernetes Migration

The architecture intentionally avoids Docker Compose-specific design decisions.

Future migration to Kubernetes should require minimal changes.

Potential Kubernetes resources include:

Deployment

Service

ConfigMap

Secret

Ingress

PersistentVolume

HorizontalPodAutoscaler

The modular monolith remains fully compatible with container orchestration platforms.

16.17 Security Considerations

Containers should follow security best practices.

Recommendations include:

- run as non-root user
- read-only filesystem where practical
- minimal base images
- image vulnerability scanning
- least privilege
- secret management through environment variables or secret stores

These practices reduce the attack surface of the deployment.

16.18 Scalability Considerations

Although the initial deployment targets a single host, the architecture supports future horizontal scaling.

Potential scaling strategies include:

Frontend

Multiple Angular instances behind Nginx

Backend

Multiple Spring Boot instances

Database

Read replicas

Connection pooling

Caching

Redis

The stateless backend architecture simplifies future scaling.

16.19 CI/CD Integration

The deployment architecture is designed for future CI/CD automation.

A typical pipeline consists of:

Commit

↓

Build

↓

Unit Tests



Integration Tests



Docker Build



Security Scan



Publish Images



Deploy



Smoke Tests

Each deployment artifact is immutable and versioned.

16.20 Docker & Deployment Summary

The deployment architecture emphasizes reproducibility, isolation and operational simplicity.

Containerization ensures consistent execution across development, testing and production environments while reducing deployment complexity.

By externalizing configuration, separating responsibilities and following production-oriented deployment practices from the beginning, the system remains well prepared for future cloud-native evolution without introducing unnecessary complexity during the initial implementation phase.

17. Logging, Monitoring & Observability

17.1 Overview

Operational excellence is an essential characteristic of production-grade software systems.

While functional correctness ensures that the application behaves as expected, observability enables developers and operators to understand how the system behaves in real-world environments.

DeutschLingoDeck is designed with observability as a first-class architectural concern rather than an afterthought.

The observability strategy consists of three complementary pillars:

- Logging
- Monitoring
- Distributed Tracing (future extension)

Together, these capabilities provide complete visibility into application behavior, system health and operational issues.

17.2 Observability Principles

The system follows several guiding principles.

Every important business operation should produce meaningful logs.

Failures should be observable before they become incidents.

Business metrics should be distinguishable from infrastructure metrics.

Logs should support troubleshooting without exposing sensitive information.

Operational visibility should require minimal manual intervention.

17.3 Logging Architecture

Logging is implemented as a centralized cross-cutting concern.

Every application component produces structured logs using a consistent format.

Logging responsibilities include:

- request lifecycle

- authentication
- business operations
- validation failures
- unexpected exceptions
- application startup
- application shutdown

Logging should never become part of business logic.

17.4 Log Levels

The application uses standardized log levels.

TRACE

Detailed diagnostic information.

Used only during deep troubleshooting.

Disabled in production.

DEBUG

Development-oriented information.

Examples include:

- SQL execution
- request mapping
- internal state transitions

Disabled in production.

INFO

Normal application behavior.

Examples:

- user login
- dictionary imported
- game completed
- application startup

Most business events belong to this level.

WARN

Recoverable problems.

Examples:

- repeated failed login attempts
- invalid upload
- expired refresh token
- deprecated API usage

Warnings indicate situations requiring attention but not immediate intervention.

ERROR

Unexpected failures.

Examples:

- database unavailable
- unhandled exception
- transaction failure
- external dependency unavailable

Errors should always contain sufficient diagnostic information.

17.5 Structured Logging

Logs should use structured JSON format.

Example:

```
{  
  "timestamp": "2026-07-02T14:52:11Z",  
  "level": "INFO",  
  "service": "game-service",  
  "correlationId": "8bcf13d2",  
  "userId": 42,
```

```
"operation": "GAME_FINISHED",  
"durationMs": 845321  
}
```

Structured logging enables efficient searching and filtering in centralized logging systems.

17.6 Correlation IDs

Every incoming request receives a Correlation ID.

The identifier accompanies the request throughout its lifecycle.

Workflow:

Incoming Request

|



Generate Correlation ID

|



Application Logs

|



Response Header

This allows developers to reconstruct the complete execution path of an individual request.

17.7 Business Event Logging

Important business events should be explicitly logged.

Examples include:

User Registered

User Logged In

Dictionary Imported

Dictionary Deleted

Game Started

Game Completed

Password Changed

Import Failed

These logs provide valuable audit information while supporting operational monitoring.

17.8 Sensitive Data Protection

Sensitive information must never appear in logs.

Examples include:

Passwords

JWT Tokens

Refresh Tokens

Authentication Headers

Personal Secrets

Raw Uploaded Files

If sensitive values must be referenced, they should be masked or omitted entirely.

Security always takes precedence over debugging convenience.

17.9 Exception Logging

Unexpected exceptions should be logged centrally.

Each exception log should include:

- timestamp

- exception type
- stack trace
- authenticated user
- endpoint
- correlation ID
- request identifier

Exceptions should never be logged multiple times.

A centralized exception handler prevents duplicate logging.

17.10 Monitoring

Monitoring continuously measures application health.

Typical monitoring targets include:

Application Availability

Request Rate

Error Rate

Average Response Time

Database Connectivity

Container Health

Monitoring provides real-time operational insight.

17.11 Health Checks

Every service exposes health information.

Spring Boot uses Actuator endpoints.

Typical health indicators include:

Application

Database

Disk Space

Memory

Configuration

Custom health indicators may later include:

Dictionary Import Service

Statistics Engine

AI Integration

Health checks should remain lightweight.

17.12 Metrics

Metrics are numerical measurements describing system behavior.

Infrastructure metrics include:

CPU Usage

Memory Usage

Disk Usage

Container Restarts

Database Connections

Business metrics include:

Games Started

Games Completed

Answers Submitted

Dictionary Imports

Average Accuracy

Average Session Duration

Separating business metrics from infrastructure metrics simplifies operational analysis.

17.13 Performance Monitoring

Performance monitoring focuses on application responsiveness.

Typical measurements include:

HTTP Response Time

Database Query Duration

Dictionary Import Duration

Game Creation Time

Answer Validation Time

Statistics Update Time

Long-running operations should generate performance warnings.

17.14 Alerting

Future production deployments should define operational alerts.

Examples include:

Database unavailable

Application unavailable

Repeated failed logins

High error rate

Slow response time

Import failures

Alerts should prioritize actionable information over notification volume.

17.15 Audit Logging

Audit logging differs from application logging.

Audit logs record security-sensitive business events.

Examples include:

Successful Login

Failed Login

Password Change

Role Assignment

Dictionary Import

Dictionary Deletion

Audit records should remain immutable.

Retention policies should satisfy organizational requirements.

17.16 Distributed Tracing (Future)

The current architecture is a modular monolith.

However, future migration to distributed systems may introduce tracing.

Example architecture:

Browser

↓

Nginx

↓

Spring Boot

↓

Database

Every request would receive a Trace ID.

Future OpenTelemetry integration would allow complete request visualization across services.

17.17 Observability Stack

The recommended future observability stack includes:

Logging

Loki

Metrics

Prometheus

Visualization

Grafana

Tracing

OpenTelemetry

This stack has become a widely adopted industry standard.

The initial implementation intentionally keeps dependencies minimal while remaining fully compatible with future integration.

17.18 Retention Policy

Different operational data should have different retention periods.

Examples:

Application Logs

30–90 days

Audit Logs

1–7 years

Metrics

12 months

Tracing Data

7–30 days

Retention policies should balance operational needs with storage costs.

17.19 Operational Dashboards

Future monitoring dashboards may include:

System Overview

Application Health

Authentication Activity

Learning Activity

Dictionary Imports

Performance Overview

Error Analysis

Business Metrics

These dashboards provide both technical and business visibility into the application.

17.20 Logging, Monitoring & Observability Summary

The observability architecture ensures that the application remains transparent, diagnosable and operationally reliable.

Structured logging, centralized exception handling, health monitoring and business metrics provide developers and operators with comprehensive insight into system behavior.

By treating observability as an architectural concern from the beginning, DeutschLingoDeck is prepared not only for local development but also for future production deployments and cloud-native environments with minimal architectural changes.

18. Testing Strategy & Quality Assurance

18.1 Overview

Software quality is not achieved solely through careful implementation. It is the result of a comprehensive engineering process that combines architecture, automated testing, static analysis, code reviews and continuous integration.

DeutschLingoDeck adopts a quality-first approach. Every architectural decision should be verifiable through automated tests, and every business rule should be protected against regressions.

Testing is considered an integral part of development rather than a separate activity performed after implementation.

18.2 Quality Objectives

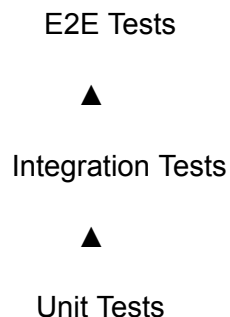
The testing strategy aims to achieve the following objectives:

- verify business correctness
- prevent regressions
- support safe refactoring
- document expected system behavior
- enable continuous delivery
- reduce production defects

Automated testing provides confidence that new functionality does not unintentionally break existing behavior.

18.3 Testing Pyramid

The project follows the classical Testing Pyramid.



The majority of tests should be unit tests.

Higher-level tests are fewer but provide broader system validation.

This balance minimizes execution time while maintaining comprehensive coverage.

18.4 Unit Testing

Unit tests verify individual classes in complete isolation.

Typical candidates include:

- Domain Services
- Validation Strategies
- Utility Classes
- Statistics Calculations
- Mappers

Unit tests should execute quickly and independently of external systems.

Dependencies are replaced by mocks or test doubles.

Every business rule should have dedicated unit tests.

18.5 Integration Testing

Integration tests verify collaboration between multiple application components.

Examples include:

- Controller → Service → Repository
- Repository → PostgreSQL
- Flyway migrations
- Spring Security configuration
- Transaction boundaries

The project uses Testcontainers to execute integration tests against a real PostgreSQL instance.

This approach significantly increases confidence compared to in-memory databases.

18.6 End-to-End Testing

End-to-End (E2E) tests verify complete user workflows from the browser to the database.

Typical scenarios include:

- User registration
- User login
- Dictionary import
- Game creation
- Answer submission
- Game completion
- Statistics retrieval

E2E tests ensure that all application layers collaborate correctly.

18.7 Test Scope by Layer

Each architectural layer has clearly defined testing responsibilities.

Controllers

Verify:

- request mapping
- response status codes
- validation
- authentication
- authorization

Controllers should not require extensive business logic tests.

Application Services

Verify:

- workflow orchestration
- transaction boundaries
- service coordination

Domain Services

Verify:

- business rules

- calculations
- validation
- state transitions

This layer receives the highest level of unit test coverage.

Repositories

Verify:

- SQL queries
- pagination
- filtering
- database mappings
- indexes
- optimistic locking

18.8 Test Data Management

Test data should be deterministic and isolated.

Every test should:

- create its own data
- avoid shared mutable state
- clean up automatically

Integration tests should never depend on execution order.

Reusable test fixtures may be introduced to reduce duplication while maintaining readability.

18.9 Test Naming

Test names should describe business behavior rather than implementation details.

Examples:

`shouldCreateDictionaryWhenImportsValid()`

`shouldRejectDuplicateDictionaryName()`

shouldCalculateCorrectStatisticsAfterAnswerSubmission()

shouldPreventAccessToAnotherUsersDictionary()

Readable test names improve maintainability and serve as executable documentation.

18.10 Test Coverage

High test coverage is desirable but should not become an objective by itself.

Meaningful tests are more valuable than artificially increasing coverage percentages.

Priority should be given to:

- business rules
- security
- validation
- transaction management
- statistics calculations

Simple getters, setters and generated code do not require explicit tests.

18.11 Performance Testing

Performance tests evaluate system responsiveness under realistic workloads.

Typical scenarios include:

- importing large dictionaries
- completing long learning sessions
- retrieving dashboard statistics
- concurrent answer submissions

Performance testing identifies bottlenecks before production deployment.

18.12 Security Testing

Security testing verifies that the application correctly enforces authentication and authorization.

Important scenarios include:

- invalid JWT
- expired JWT
- refresh token reuse
- IDOR attacks
- unauthorized API access
- invalid file uploads
- SQL injection attempts
- XSS prevention

Security tests are considered mandatory for production readiness.

18.13 Frontend Testing

Frontend tests verify Angular components and services.

Examples include:

Component Tests

- rendering
- user interaction
- Signals updates
- Reactive Forms

Service Tests

- HTTP communication
- state management
- interceptors
- route guards

UI behavior should remain deterministic and independent of backend implementation.

18.14 Continuous Integration

Every code change should automatically trigger the testing pipeline.

Recommended pipeline:

Source Code

|



Static Analysis

|



Unit Tests

|



Integration Tests

|



Build

|



Docker Image

Code should never be merged if mandatory quality checks fail.

18.15 Static Code Analysis

Automated code analysis complements traditional testing.

Recommended tools include:

Backend

- Checkstyle
- SpotBugs
- PMD

Frontend

- ESLint
- Angular ESLint

General

- SonarQube

Static analysis identifies problems before runtime.

18.16 Code Review

Every Pull Request should undergo peer review.

Review criteria include:

- architectural consistency
- readability
- security
- performance
- naming
- test quality
- documentation

Code reviews are intended to improve software quality rather than simply approve changes.

18.17 Definition of Done

A feature is considered complete only when all of the following conditions are satisfied:

- implementation completed
- unit tests passing
- integration tests passing
- code reviewed
- documentation updated
- static analysis successful
- CI pipeline successful

Completion is therefore defined by quality criteria rather than implementation alone.

18.18 Future Testing Enhancements

Future versions may introduce additional testing capabilities.

Examples include:

- contract testing
- mutation testing
- accessibility testing

- visual regression testing
- load testing
- chaos engineering
- resilience testing

These techniques further strengthen application reliability as the project evolves.

18.19 Quality Metrics

The project should monitor engineering quality metrics.

Examples include:

- build success rate
- deployment frequency
- defect rate
- code coverage
- test execution time
- mean time to recovery (MTTR)
- change failure rate

These metrics provide objective insight into development quality and team performance.

18.20 Testing Strategy Summary

The testing strategy establishes quality as a continuous engineering practice rather than a final verification step.

By combining automated testing, static analysis, code reviews and continuous integration, the project achieves a high level of confidence while supporting safe refactoring and long-term maintainability.

Every architectural layer is tested according to its responsibility, ensuring that business rules, infrastructure and user-facing functionality remain reliable throughout the lifecycle of the application.

19. Development Standards & Engineering Guidelines

19.1 Overview

Software architecture alone does not guarantee a maintainable system. Long-term quality depends equally on consistent engineering practices followed by every developer contributing to the project.

This chapter defines the engineering standards that govern implementation, code organization, collaboration and software quality throughout the lifecycle of DeutschLingoDeck.

The objective is to ensure that every contribution follows the same architectural principles, coding conventions and quality expectations, regardless of the developer implementing the feature.

These guidelines are considered part of the system architecture and should be followed consistently.

19.2 Engineering Principles

Every implementation should follow the following engineering principles.

Simplicity

Prefer the simplest solution that satisfies the business requirement.

Avoid unnecessary abstractions.

Avoid speculative generalization.

Readability

Code is written primarily for humans.

Readable code is preferred over clever code.

A developer unfamiliar with the project should understand a class after reading it once.

Maintainability

Business rules should be easy to locate.

Every class should have a single responsibility.

Dependencies should remain explicit.

Testability

Every business rule should be independently testable.

Hidden dependencies should be avoided.

Business logic should never depend directly on infrastructure.

Consistency

The project should feel as though it was written by a single developer.

Naming, package organization and architectural patterns should remain consistent across the entire codebase.

19.3 SOLID Principles

All business code should follow the SOLID principles.

Single Responsibility Principle

Every class should have one reason to change.

Examples:

Good:

AnswerValidationService

CardSelectionService

StatisticsAggregationService

Avoid:

GameService

containing:

- validation
- persistence
- statistics
- authentication
- reporting

Open/Closed Principle

Components should be open for extension but closed for modification.

New validation strategies should be introduced through new implementations rather than modifying existing classes.

Liskov Substitution Principle

Derived implementations must behave consistently with their abstractions.

Every implementation of a strategy interface should remain interchangeable.

Interface Segregation Principle

Prefer several focused interfaces over one large interface.

Dependency Inversion Principle

High-level business logic should depend on abstractions rather than concrete implementations.

19.4 Package Organization

Both frontend and backend follow feature-based organization.

Example:

dictionary/

controller

service

repository

entity

dto

mapper

validator

Technical packages spanning the entire project should be avoided unless they truly represent shared infrastructure.

19.5 Naming Conventions

Names should describe business intent rather than implementation.

Examples:

Good:

DictionaryImportService

AnswerValidationStrategy

StatisticsAggregator

GameSummaryResponse

Avoid:

Utils

Helper

Manager

Processor

CommonService

Generic names reduce readability.

19.6 Class Size

Classes should remain small and focused.

Recommended guidelines:

Controller

< 150 lines

Service

< 300 lines

Domain Service

< 200 lines

Methods

Ideally under 30 lines

Large classes usually indicate multiple responsibilities.

19.7 Method Design

Methods should perform one clearly defined task.

Good methods:

- short
- descriptive
- deterministic
- easy to test

Avoid methods containing multiple unrelated business operations.

Prefer extracting private helper methods rather than deeply nested logic.

19.8 Design Patterns

Only patterns solving actual problems should be introduced.

Patterns intentionally used in this project include:

Strategy Pattern

Factory Pattern

Builder Pattern

Dependency Injection

Repository Pattern

Mapper Pattern

Pipeline Pattern

Facade Pattern

Template Method (where appropriate)

Patterns should improve clarity rather than increase abstraction.

19.9 Clean Code Guidelines

Developers should follow Clean Code principles.

Examples include:

Meaningful names

Small functions

No duplicated logic

Limited nesting

Early returns

Immutable DTOs

Constructor injection

Avoid magic numbers

Avoid commented-out code

Readable code is preferred over compact code.

19.10 Exception Handling

Exceptions represent exceptional situations.

Business validation should not rely on exceptions for normal control flow.

Exception hierarchy should remain consistent.

Custom exceptions should describe business problems rather than technical failures.

19.11 Logging Guidelines

Logging should provide operational value.

Do log:

Business events

Unexpected failures

Security events

Performance warnings

Do not log:

Passwords

JWT tokens

Personal secrets

Large payloads

Sensitive data

Logs should help diagnose problems rather than generate unnecessary noise.

19.12 Dependency Rules

Dependencies should always point toward lower architectural layers.

Allowed:

Controller

↓

Application Service

↓

Domain Service

↓

Repository

Forbidden:

Repository → Controller

Entity → DTO

Controller → Repository

Feature → Feature Repository

Dependencies should never violate module boundaries.

19.13 Git Workflow

The project follows Git Flow with slight simplifications.

Primary branches:

main

develop

Feature branches:

feature/*

Bug fixes:

bugfix/*

Hotfixes:

hotfix/*

Releases:

release/*

Every feature should be developed in an isolated branch.

19.14 Commit Message Convention

The project adopts Conventional Commits.

Examples:

feat(dictionary): implement Excel import

fix(game): prevent duplicate answer submission

refactor(validation): introduce Strategy pattern

test(statistics): add aggregation tests

docs(api): update REST documentation

Consistent commit messages improve project history and support automated release generation.

19.15 Pull Request Guidelines

Every Pull Request should contain:

Purpose

Implementation Summary

Testing Summary

Breaking Changes

Screenshots (if applicable)

Related Issue

Pull Requests should remain focused on a single feature whenever possible.

19.16 Code Review Checklist

Reviewers should verify:

Architecture

Business correctness

Security

Readability

Naming

Performance

Transaction boundaries

Validation

Tests

Documentation

The objective is collaborative improvement rather than approval.

19.17 Documentation Standards

Public APIs should be self-explanatory through naming.

Complex business rules should be documented.

Architecture decisions belong in ADR documents rather than source code comments.

Comments should explain **why**, not **what**.

Good:

Using optimistic locking to prevent lost updates.

Avoid:

Increment counter by one.

19.18 Refactoring Guidelines

Developers are encouraged to improve existing code whenever appropriate.

Refactoring should:

- preserve behavior
- improve readability
- simplify architecture
- reduce duplication

Large refactorings should be performed separately from feature development whenever practical.

19.19 Technical Debt

Technical debt should be visible rather than hidden.

Known limitations should be documented.

Temporary workarounds should contain:

- justification
- expected removal strategy

- related issue reference

Long-term architectural quality takes precedence over short-term delivery speed.

19.20 Engineering Culture

The project promotes an engineering culture based on:

Continuous Learning

Constructive Feedback

Collective Ownership

Automation

Knowledge Sharing

Architectural Consistency

Developers are encouraged to challenge existing solutions when better alternatives exist, provided that architectural consistency is maintained.

19.21 Definition of Ready

A development task is considered ready for implementation when:

- business requirements are clear
- acceptance criteria are defined
- UI expectations are known
- API contract exists
- dependencies are identified
- architectural impact has been reviewed

Beginning implementation before these conditions are met increases project risk.

19.22 Engineering Handbook Summary

This chapter establishes the engineering standards that guide implementation throughout the project.

By combining architectural discipline, Clean Code principles, SOLID design, consistent naming conventions, structured code reviews and standardized development workflows, the project ensures long-term maintainability and high software quality.

These standards are intended to remain stable throughout the lifetime of the application and should be considered mandatory engineering practices rather than optional recommendations.

20. Future Roadmap & Architectural Evolution

20.1 Overview

A well-designed software architecture should not only satisfy current business requirements but also provide a clear path for future evolution.

DeutschLingoDeck has been intentionally designed as a modular, production-oriented system capable of growing significantly without requiring fundamental architectural redesign.

The initial implementation focuses on delivering a stable, maintainable and extensible learning platform. Future enhancements should build upon the existing architecture while preserving its core principles.

Architectural evolution should always favor incremental improvements over disruptive rewrites.

20.2 Evolution Principles

Future development should follow several guiding principles.

Preserve Existing Architecture

New features should integrate naturally into the existing modular structure.

Whenever possible, existing abstractions should be extended rather than replaced.

Backward Compatibility

Existing APIs should remain stable.

Breaking changes should be introduced only when absolutely necessary.

Versioning should be used whenever backward compatibility cannot be preserved.

Incremental Complexity

Architectural complexity should grow only when justified by measurable business value.

Features should not be introduced merely because they are technologically interesting.

Domain-Driven Growth

New modules should represent new business capabilities rather than technical concepts.

Architecture should continue evolving around the business domain.

20.3 Phase 1 – Production MVP

The first production version focuses on core learning functionality.

Scope includes:

User Management

Authentication

Dictionary Import

Game Engine

Answer Validation

Statistics Engine

Dashboard

Docker Deployment

Monitoring

Automated Testing

The primary objective is to establish a stable architectural foundation.

20.4 Phase 2 – Enhanced Learning Experience

The second phase introduces richer learning functionality.

Examples include:

Custom Game Modes

Random Card Selection

Card Filtering

Difficulty Levels

Bookmarks

Favorite Cards

Learning Goals

Daily Challenges

Learning Streaks

Achievements

All these features fit naturally into the existing Game Engine and Statistics Engine.

20.5 Phase 3 – Adaptive Learning

The third phase introduces adaptive learning algorithms.

Potential features include:

Spaced Repetition

Adaptive Card Scheduling

Memory Strength Calculation

Review Intervals

Confidence Levels

Learning Prioritization

Weak Topic Detection

These capabilities can be implemented by extending the existing Card Selection Service and Statistics Engine.

No changes to the public REST API are required.

20.6 Phase 4 – Artificial Intelligence

Artificial Intelligence becomes an optional enhancement rather than a replacement for deterministic business logic.

Potential AI capabilities include:

Grammar Evaluation

Sentence Correction

Alternative Answer Suggestions

Explanation Generation

Vocabulary Recommendations

Personal Learning Coach

Conversation Practice

Adaptive Study Plans

LLM integration should remain isolated behind dedicated interfaces.

Business logic must never depend directly on a specific AI provider.

20.7 Phase 5 – Voice-Based Learning

Future versions may introduce speech-based interaction.

Potential capabilities include:

Speech Recognition

Pronunciation Evaluation

Voice Commands

Text-to-Speech

Conversation Mode

Listening Exercises

Pronunciation Feedback

The existing validation architecture already supports multiple validation strategies, making voice validation a natural extension.

20.8 Multi-Language Support

Although the initial version focuses on Croatian-to-German learning, the architecture intentionally avoids language-specific assumptions.

Future versions should support:

English

French

Italian

Spanish

Dutch

Japanese

Multiple language pairs

The domain model already separates source and target languages, allowing this evolution without structural changes.

20.9 Mobile Applications

The REST API is platform-independent.

Future client applications may include:

Android

iOS

Desktop

Progressive Web App (PWA)

Smart TV

The backend architecture requires no modifications to support additional clients.

20.10 Offline Learning

Offline support may be introduced through Progressive Web Application technologies.

Potential capabilities include:

Offline Dictionary Access

Offline Game Sessions

Background Synchronization

Conflict Resolution

Cached Statistics

Offline operation should remain transparent to the user whenever possible.

20.11 Collaboration Features

Future versions may introduce collaborative capabilities.

Examples include:

Dictionary Sharing

Public Dictionaries

Collaborative Editing

Comments

Ratings

Import Marketplace

Version History

Ownership rules remain unchanged.

Sharing becomes an additional permission layer rather than replacing ownership.

20.12 Gamification

Gamification should encourage long-term engagement.

Potential features include:

Experience Points

Levels

Achievements

Badges

Leaderboards

Daily Rewards

Weekly Challenges

Learning Seasons

Gamification should complement learning rather than distract from it.

20.13 Analytics

Future analytical capabilities may include:

Learning Trends

Vocabulary Growth

Weak Topic Analysis

Grammar Error Distribution

Response Time Analysis

Daily Learning Heat Maps

Retention Curves

Predictive Learning Models

The historical event model already contains all required information.

20.14 Event-Driven Architecture

Although the current implementation is a modular monolith, future growth may justify asynchronous event processing.

Potential domain events include:

DictionaryImported

GameStarted

AnswerSubmitted

AnswerValidated

GameFinished

StatisticsUpdated

Event publication should remain internal initially.

Future migration to Apache Kafka or RabbitMQ should require only infrastructure changes.

20.15 Cloud-Native Deployment

Future deployments may migrate from Docker Compose to Kubernetes.

Potential infrastructure additions include:

Kubernetes

Helm

Horizontal Pod Autoscaler

Ingress Controller

Managed PostgreSQL

Object Storage

Secret Management

Cloud Monitoring

The current containerized architecture intentionally prepares for this evolution.

20.16 Microservices Evaluation

The current architecture intentionally avoids premature decomposition into microservices.

Migration should only be considered if measurable business requirements emerge.

Potential extraction candidates include:

Authentication

Dictionary Import

Statistics

AI Integration

Notification Service

Extraction should preserve existing API contracts.

Microservices are considered an operational optimization rather than an architectural objective.

20.17 Engineering Evolution

As the project grows, engineering practices should evolve alongside the architecture.

Potential future improvements include:

Contract Testing

Mutation Testing

Performance Benchmarking

Automated Dependency Updates

Continuous Security Scanning

Infrastructure as Code

Release Automation

Developer Portals

Engineering maturity should increase gradually without sacrificing simplicity.

20.18 Architectural Stability

Certain architectural principles should remain stable regardless of future features.

These include:

Modular Architecture

Feature-Based Organization

Stateless Backend

Ownership-Based Authorization

Immutable Historical Events

API First

Database as Source of Truth

Single Responsibility Principle

These principles form the architectural identity of the system.

20.19 Long-Term Vision

The long-term vision of DeutschLingoDeck extends beyond a traditional flashcard application.

The architecture is intended to support an intelligent learning platform capable of adapting to individual users, integrating artificial intelligence and supporting multiple interaction models while maintaining a clean, maintainable and production-ready codebase.

Future innovation should build upon the existing foundation rather than replacing it.

Architectural consistency remains more valuable than rapid feature expansion.

20.20 Architectural Evolution Summary

The architecture presented throughout this document has been designed with long-term evolution in mind.

By combining modular design, strong separation of concerns, feature-oriented organization and production-ready engineering practices, the system provides a stable foundation for future growth.

New capabilities—including AI-assisted learning, adaptive algorithms, voice interaction, cloud-native deployment and advanced analytics—can be introduced incrementally without compromising the integrity of the existing architecture.

The ultimate objective is not simply to build a functional application, but to establish a sustainable software platform capable of evolving over many years while remaining understandable, maintainable and technically robust.